LLM 核心结构与训练：Q/K 归一化、RMSNorm、GQA、RoPE、YaRN、KV Cache、FFN、残差连接、MoE、SFT 与 LoRA

> 适合作为 Transformer / LLM 原理学习笔记。
> 核心内容：**Q/K 归一化、RMSNorm、GQA、RoPE、YaRN、KV Cache、FFN、残差连接**
> 可选扩展：**MoE、LoRA 原理**
> 训练部分：**SFT：数据、损失与训练流程（第 21 章）**、**LoRA：梯度更新与训练实践（第 22 章）**

1. 总览：这些技术分别解决什么问题？

在标准 Transformer 中，一层通常包含：

1. 输入 Hidden States
2. Self-Attention
3. FFN / MLP
4. 残差连接与归一化

Self-Attention 的核心是：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

然后计算注意力分数：

$$A = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

最后：

$$O = AV$$

下面把基本组件与常见改进放在一起，看看它们各自解决什么问题：

技术	主要解决的问题
Q/K 归一化	防止注意力分数过大，提高训练稳定性
RMSNorm	按均方根调整特征尺度，常用于 Attention / FFN 前
残差连接	保留原隐藏状态并叠加分支更新，提供直接的信息与梯度路径
FFN	对每个 Token 的特征做非线性变换，常用升维—激活—降维结构
GQA	减少 K/V Head 数量，降低 KV Cache 显存和推理开销
RoPE	向 Q/K 注入位置信息，使 Attention 感知 Token 相对位置
YaRN	调整 RoPE 的频率与注意力尺度，帮助模型适应更长上下文

技术	主要解决的问题
KV Cache	推理时缓存历史 K/V，避免重复计算历史 Token
MoE	用多个 Expert 扩大模型容量，但每次只激活少量参数
SFT	用输入与参考回答的示范数据，训练模型遵循指令、完成任务
LoRA	用低秩矩阵进行参数高效微调，避免更新完整模型参数

以采用 Pre-Norm 的 Transformer 层为例，可以把它们放进下面的流程：

```
Token → Embedding → 当前层输入 X
                        │
                        ▼
                    RMSNorm 1
                        │
                        ▼
                    Attention
                    ├── Q/K 归一化（可选）：控制 Q/K 数值
                    ├── RoPE：给 Q/K 加位置信息
                    │   └── YaRN（可选）：调整长上下文的频率与尺度
                    ├── GQA：减少 K/V Head
                    └── KV Cache：缓存历史 K/V
                        │
                        ▼
                残差相加：X + Attention 输出 → H
                        │
                        ▼
                    RMSNorm 2
                        │
                        ▼
                     FFN / MoE：加工特征
                        │
                        ▼
                残差相加：H + FFN 输出 → 下一层
```

LoRA 则可以插入：

- Attention 的 W_Q, W_K, W_V, W_O
- FFN 的线性层
- 或其他权重矩阵

用于微调。SFT 定义示范数据上的训练目标，LoRA 定义参数更新方式；两者可以组合为一次 LoRA SFT 训练，详见第 21、22 章。

---

2. Q/K 归一化（QK Normalization）

2.1 为什么 Attention 中要有 Q 和 K?

对于一个 Token 的 Query：

$$q_i$$

以及另一个 Token 的 Key：

$$k_j$$

Attention 使用点积：

$$q_i^T k_j$$

衡量二者的匹配程度。

例如：

```
Token 1: "猫"
Token 2: "喜欢"
Token 3: "鱼"
```

当模型计算“猫”应该关注谁时，会计算：

$$q_{猫}^T k_{猫}$$

$$q_{猫}^T k_{喜欢}$$

$$q_{猫}^T k_{鱼}$$

如果：

```
猫 -> 猫      0.5
猫 -> 喜欢    1.2
猫 -> 鱼      3.6
```

Softmax 后，“猫”会更多关注“鱼”。

2.2 为什么标准 Attention 要除以 $\sqrt{d}$?

标准公式为：

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

如果：

$$q_i,k_i \sim N(0,1)$$

并且：

$$q^T k = \sum_{i=1}^{d} q_i k_i$$

那么随着维度 d 增大，点积的方差也会增大。

因此需要：

$$\frac{q^T k}{\sqrt{d}}$$

控制 Attention Logit 的尺度。

2.3 仅仅除以 $\sqrt{d}$ 为什么还不够？

训练过程中，Q 和 K 的模长可能不断变大。

例如最开始：

$$q = [1, 1]$$

$$k = [1, 1]$$

那么：

$$q^T k = 2$$

但训练一段时间后可能变成：

$$q = [20, 20]$$

$$k = [20, 20]$$

此时：

$$q^T k = 800$$

即使除以：

$$\sqrt{2}$$

结果仍然很大：

$$\frac{800}{\sqrt{2}} \approx 565.7$$

Softmax 会变得非常尖锐。

例如：

$$\mathrm{softmax}([1, 2, 3])$$

大约为：

$$[0.09, 0.24, 0.67]$$

但如果变成：

$$\mathrm{softmax}([100, 200, 300])$$

结果几乎是：

$$[0, 0, 1]$$

这会产生两个问题：

1. Attention 过度集中在少量 Token。
2. Softmax 梯度容易变得很小，训练不稳定。

2.4 Q/K 归一化的核心思想

一种常见做法是在计算 Attention 之前，对 Q 和 K 做归一化：

$$\hat{Q} = \mathrm{Norm}(Q)$$

$$\hat{K} = \mathrm{Norm}(K)$$

然后：

$$A = \mathrm{softmax}\left(\frac{\hat{Q}\hat{K}^T}{\sqrt{d}}\right)$$

具体模型可能使用：

- LayerNorm
- RMSNorm
- L2 Normalization

RMSNorm 的原理和手算例子见第 3 章。下面的 2.5、2.6 节先以 L2 归一化为例；不同方法对向量长度的约束不同，不能把单位长度结论直接套到 RMSNorm 上。

2.5 以 L2 归一化为例

对于：

$$q = [3, 4]$$

它的模长：

$$\|q\|_2 = \sqrt{3^2 + 4^2} = 5$$

归一化之后：

$$\hat{q} = \left[\frac{3}{5}, \frac{4}{5}\right] = [0.6, 0.8]$$

如果：

$$k = [6, 8]$$

那么：

$$\hat{k} = [0.6, 0.8]$$

原始点积：

$$q^T k = 3 \times 6 + 4 \times 8 = 50$$

归一化后：

$$\hat{q}^T \hat{k} = 0.6^2 + 0.8^2 = 1$$

因此 Attention 的相似度更多反映：

> Q 与 K 的方向是否相似

而不是：

> Q/K 的模长到底有多大。

2.6 直观理解

可以将 Q/K 想象成两根箭头。

```
没有归一化：

        K
       /
      /
     /
----/----------> Q

点积同时受：
1. 夹角
2. 向量长度
影响
```

而采用 L2 归一化以后：

```
Q 和 K 都缩放成单位长度

Attention 更主要由方向决定。
```

因此 Q/K Normalization 可以抑制：

$$\|Q\| \uparrow, \qquad \|K\| \uparrow$$

造成 Attention Logit 爆炸。

2.7 一句话总结

> **Q/K 归一化的核心不是减少计算量，而是控制 Attention Logit 的尺度，提高训练稳定性。**

---

3. RMSNorm：先把特征的整体尺度调稳

> 学习目标：能手算一次 RMSNorm，知道它与 LayerNorm、L2 归一化的区别，并能在 Transformer 代码中认出它的位置。
>
> 建议先读 3.1 ~ 3.5 建立直觉，再用后面的维度例子、流程图和代码把概念串起来。

3.1 为什么需要 RMSNorm？从“同样的形状，不同的大小”讲起

在 Transformer 中，每个 Token 都有一个 Hidden State 向量。为了方便手算，假设它只有 4 个维度：

$$x = [1, 2, 2, 1]$$

另一个向量是：

$$x' = [10, 20, 20, 10] = 10x$$

两个向量中各分量的比例一样，但第二个向量整体放大了 10 倍。经过相同的无偏置线性层后，这种放大也会传递下去：

$$(10x)W = 10(xW)$$

如果这些值进入 Q/K 点积、非线性函数等运算，数值大小就会影响后续计算。前一章中 Q/K 模长增大导致 Attention 分数变大的例子，就是一种表现。

可以把这种情况类比成两段内容相同、录音音量不同的声音：我们希望先把整体音量调到相近水平，再交给后面的模块处理。

RMSNorm 的思路是：先用一个数衡量当前 Token 向量的整体大小，再用它去除这个向量的每个分量。

这会减少后续模块输入尺度的波动，但不代表两个大小不同的向量在所有任务中都表达同样的信息。这里只是用它们说明归一化的数学作用。

3.2 RMS 是什么？为什么不用普通平均值？

RMS 是 **Root Mean Square**，中文是“均方根”。按计算顺序理解，就是：

> 每个数平方 → 求平均 → 开平方

对于一个有 d 个分量的向量：

$$x = [x_1, x_2, \ldots, x_d]$$

它的 RMS 为：

$$\mathrm{RMS}(x) = \sqrt{\frac{x_1^2 + x_2^2 + \cdots + x_d^2}{d}}$$

为什么先平方？因为特征可以有正有负，普通平均值会让正负抵消。

例如：

$$x = [3, -3, 3, -3]$$

普通平均值是：

$$\frac{3 - 3 + 3 - 3}{4} = 0$$

但每个分量的绝对值明明都是 3，不能说这个向量“没有大小”。用 RMS 就可以得到：

$$\mathrm{RMS}(x) = \sqrt{\frac{9 + 9 + 9 + 9}{4}} = 3$$

平方让正负数都对大小有贡献；求平均让统计量考虑维度数量；最后开平方，把平方后的量还原到与输入相同的量纲。

注意：**平方只用于计算分母，分子仍然是原来的数。** 所以归一化不会把所有负数变成正数。

3.3 完整公式：归一化，再乘可学习参数

常见 RMSNorm 的逐维公式为：

$$\boxed{y_i = \gamma_i \frac{x_i}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \epsilon}}}$$

也可以拆成两步：

$$z_i = \frac{x_i}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \epsilon}}, \qquad y_i = \gamma_i z_i$$

符号	含义	怎么理解
x_i	输入向量的第 i 个分量	当前 Token 的一个特征值
d	参与归一化的维度数	此例中是 4
ϵ	一个很小的正数	防止分母为零，改善数值稳定性

符号	含义	怎么理解
z_i	归一化后的中间值	先统一整体尺度
γ_i	第 i 维的可学习缩放参数	再让模型调整各维的幅度
y_i	最终输出	交给下一步计算的特征

同一个 Token 的所有分量共用一个分母；不同维度的缩放参数可以不同。常见实现将这些参数初始化为 1，并且不额外加偏置。公式和参数定义可对照 PyTorch RMSNorm 文档。

接下来的手算若无特别说明，都先忽略很小的 ϵ，并令所有 $\gamma_i = 1$。 代码中会保留正的 ϵ。

3.4 完整手算：把一组数字走到底

仍然使用：

$$x = [1, 2, 2, 1]$$

第一步：每个分量平方。

$$x^2 = [1, 4, 4, 1]$$

第二步：求平方的平均值。

$$m = \frac{1 + 4 + 4 + 1}{4} = \frac{10}{4} = 2.5$$

第三步：开平方，得到整体尺度。

$$\mathrm{RMS}(x) = \sqrt{2.5} \approx 1.5811$$

第四步：每个原始分量除以这个数。

$$z = \left[\frac{1}{\sqrt{2.5}}, \frac{2}{\sqrt{2.5}}, \frac{2}{\sqrt{2.5}}, \frac{1}{\sqrt{2.5}}\right] \approx [0.6325, 1.2649, 1.2649, 0.6325]$$

这里有三个值得观察的结果：

- 原来的 2 仍然是原来的 1 的两倍，分量比例没有改变。
- 输出中有数大于 1，因此 RMSNorm 并不是把每个数限制到 0 ~ 1。
- 输出的 RMS 等于 1，但输出的均值不等于 0。

最后一条可以直接验算：

$$\mathrm{RMS}(z) = \sqrt{\frac{0.4 + 1.6 + 1.6 + 0.4}{4}} = 1$$

再把输入整体放大 10 倍：

$$\mathrm{RMS}([10, 20, 20, 10]) = \sqrt{250} = 10\sqrt{2.5}$$

所以归一化后的结果仍然是：

$$\frac{[10, 20, 20, 10]}{10\sqrt{2.5}} = \frac{[1, 2, 2, 1]}{\sqrt{2.5}}$$

这说明：**忽略 ϵ 时，输入整体乘以一个正数，RMSNorm 的结果不变。** 保留 ϵ 后，通常只能说近似不变；当输入非常小时，差异会变明显。

“正数”这个条件也很关键：如果整个输入乘以 -1，输出也会反号。

3.5 已经归一化了，为什么还要乘 gamma?

统一整体尺度，是为了让输入更容易处理；但不同特征维度适合的幅度，不一定完全相同。

因此 RMSNorm 给每一维配一个可以训练的缩放参数：

$$\gamma = [\gamma_1, \gamma_2, \ldots, \gamma_d]$$

假设训练后得到：

$$\gamma = [1, 2, 0.5, 1]$$

沿用上一节的中间结果：

$$z \approx [0.6325, 1.2649, 1.2649, 0.6325]$$

最终输出是逐元素相乘：

$$y = \gamma \odot z \approx [0.6325, 2.5298, 0.6325, 0.6325]$$

第二维被放大，第三维被缩小。这是模型通过训练学到的调整；这里的数字只是演示，并不表示某一维固定对应某个语义。

要区分两个阶段：

```
输入 x
  ↓
所有维度除以同一个分母
  ↓
中间结果 z：统一整体尺度
  ↓
每个维度乘自己的 gamma
  ↓
最终输出 y：允许模型学习适合的各维幅度
```

因此，**“RMSNorm 保持方向”“输出 RMS 一定为 1”都需要条件。** 只看除以公共分母这一步，方向保持不变；乘上不同的 gamma 后，方向和整体尺度都可能改变。

gamma 也不是每个 Token 各自训练一套。在同一个 RMSNorm 层中，不同 Token 通常共享这 d 个参数；不同层通常有各自的参数。

3.6 和 LayerNorm、L2 归一化有什么区别?

先用同一组数字比较 LayerNorm

LayerNorm 会先减均值，再除以标准差，最后进行可学习的缩放和偏移：

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i-\mu)^2$$

$$\mathrm{LayerNorm}(x)_i = \gamma_i \frac{x_i-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta_i$$

下面比较时忽略 ϵ，令 gamma 全为 1、LayerNorm 的 beta 全为 0。

对于同一个输入：

$$x = [1, 2, 2, 1]$$

先求均值：

$$\mu = \frac{1+2+2+1}{4} = 1.5$$

再减均值：

$$x-\mu = [-0.5, 0.5, 0.5, -0.5]$$

标准差是：

$$\sigma = \sqrt{\frac{0.25+0.25+0.25+0.25}{4}} = 0.5$$

所以 LayerNorm 得到：

$$[-1, 1, 1, -1]$$

而 RMSNorm 得到：

$$[0.6325, 1.2649, 1.2649, 0.6325]$$

区别已经很直观：LayerNorm 的这一步会把向量的均值移到 0；RMSNorm 不减均值，只按均方根缩放。

再看看“全部加 10”会怎样

把输入改成：

$$x + 10 = [11, 12, 12, 11]$$

LayerNorm 减去新的均值 11.5 后，仍然得到同样的中心化向量，因此归一化结果仍然是：

$$[-1, 1, 1, -1]$$

RMSNorm 则得到：

$$\frac{[11, 12, 12, 11]}{\sqrt{132.5}} \approx [0.9556, 1.0425, 1.0425, 0.9556]$$

这说明两件不同的事：

- **整体乘一个正数**：RMSNorm 在忽略 ϵ 时结果不变。
- **每个分量加同一个数**：RMSNorm 的结果一般会改变。

不要把“缩放”与“平移”混为一谈。

RMS 和标准差之间的关系

展开平方，可以得到：

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d} x_i^2 - \mu^2$$

因此：

$$\boxed{\mathrm{RMS}(x)^2 = \sigma^2 + \mu^2}$$

RMS 衡量的是数值相对 **0** 的大小，标准差衡量的是数值相对 **均值** 的波动。

如果一个非零向量的均值恰好为 0，那么 RMS 就等于标准差；此时使用相同的 gamma、相同的 ϵ，并让 LayerNorm 的 beta 为 0，两者输出相同。

与上一章的 L2 归一化比较

L2 归一化除以向量的欧氏长度：

$$\|x\|_2 = \sqrt{\sum_{i=1}^{d} x_i^2}$$

RMS 则还要除以维度数再开平方：

$$\mathrm{RMS}(x) = \frac{\|x\|_2}{\sqrt{d}}$$

所以，忽略 ϵ 且 gamma 全为 1 时：

$$\mathrm{RMSNorm}(x) = \sqrt{d}\,\frac{x}{\|x\|_2}$$

比较项	RMSNorm	LayerNorm	L2 归一化
是否先减均值	否	是	否
除以什么	均方根	标准差	欧氏长度
不计仿射参数和稳定项时的结果	RMS 为 1	均值为 0、方差为 1	L2 长度为 1
对 $[1,2,2,1]$	$[0.6325, 1.2649, 1.2649, 0.6325]$	$[-1,1,1,-1]$	$[0.3162, 0.6325, 0.6325, 0.3162]$

比较项	RMSNorm	LayerNorm	L2 归一化
的结果			
常见可学习参数	逐维 gamma	逐维 gamma、beta	基础运算本身通常没有

表中的理想性质要求相应分母非零。带 ϵ、带可学习参数时，不能直接照搬这些等式。

对 d 维非零向量，理想 RMSNorm 后的 L2 长度是 $\sqrt{d}$，并不是 1。 这也是它与上一章 L2 归一化最容易混淆的地方。

3.7 到底沿哪个维度归一化？每个 Token 自己算

在常见的 LLM 实现中，Hidden States 的形状是：

$$X \in \mathbb{R}^{B \times T \times D}$$

维度	含义
B	一批有多少条序列
T	每条序列有多少个 Token
D	每个 Token 有多少维特征

如果 RMSNorm 配置为归一化最后的 D 维，就会对每个 Token 的特征向量单独计算 RMS。

假设 B=1、T=2、D=4：

```
一条序列，共两个 Token：

Token A → [ 1,  2,  2,  1] → 自己算 RMS → 1.5811
Token B → [10, 20, 20, 10] → 自己算 RMS → 15.8114

A 的四个数除以 A 的分母。
B 的四个数除以 B 的分母。
两个 Token 共享同一层的 gamma。
```

这里会算出 **两个分母**，不是把 8 个数放在一起算一个分母。

相应操作的形状变化是：

```
输入 X                          [B, T, D]
平方后，沿最后一维求平均        [B, T, 1]
开平方，作为分母                [B, T, 1]
广播除法，再乘 gamma            [B, T, D]
```

对于 B=2、T=3、D=4 的输入，会计算 6 个 RMS 值；这个 RMSNorm 层仍然只需要 4 个 gamma 参数。

这个设置不跨 Token 混合统计量，也不依赖同一个 batch 中的其他样本，不需要 BatchNorm 那样的运行均值或运行方差。单看这个归一化操作，新加入一个 Token 不会改变历史 Token 已有的归一化结果。

3.8 RMSNorm 在 Transformer 中放在哪里?

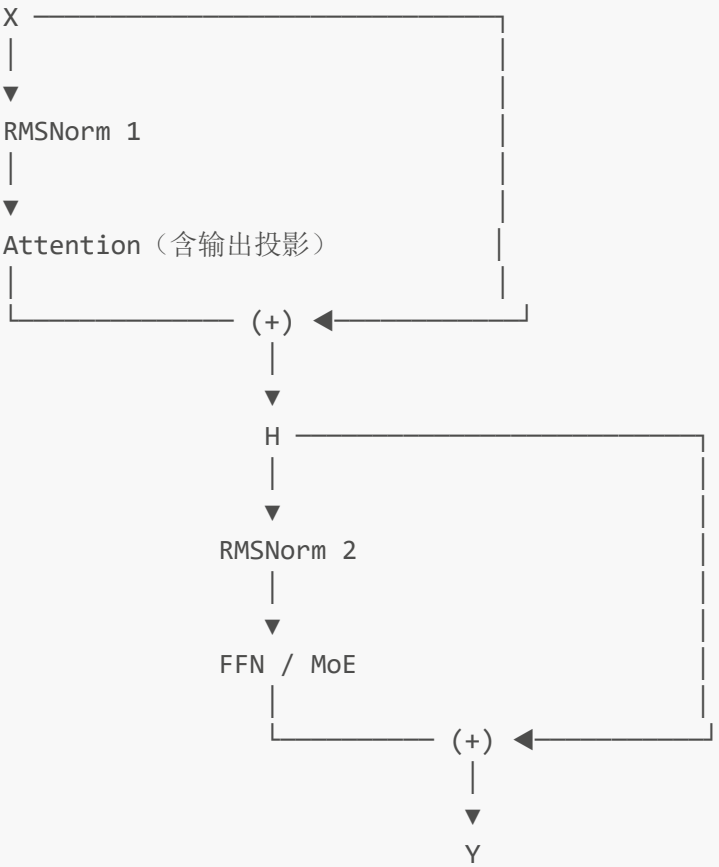
一种常见结构是 **Pre-Norm：先归一化，再进入 Attention 或 FFN 分支，随后加回残差。** 例如 Meta 的 Llama 参考实现 就在 Attention 和 FFN 前分别使用 RMSNorm。

一层可以写成：

$$H = X + \mathrm{Attention}\big(\mathrm{RMSNorm}_1(X)\big)$$

$$Y = H + \mathrm{FFN}\big(\mathrm{RMSNorm}_2(H)\big)$$

这里 Attention 包含 Q/K/V 投影、注意力计算和输出投影；FFN 可以按模型结构换成 MoE。两个 RMSNorm 通常有各自的 gamma。FFN 的矩阵手算见第 10 章，残差连接的原理与梯度例子见第 11 章。



注意残差加回的是原来的 X 或 H，不能误写成“把归一化后的向量加回去”。

仍以 X=$[1, 2, 2, 1]$ 为例。假设 Attention 分支经过输出投影后算出了：

$$a = [0.1, -0.2, 0.3, 0]$$

那么第一次残差相加得到：

$$H = X + a = [1.1, 1.8, 2.3, 1]$$

这里 a 是为了演示残差相加而假设的输出，不是仅凭 RMSNorm 就能推算出来的。

随后 RMSNorm 2 要根据 **新的 H** 重新计算分母，再把结果送入 FFN。归一化发生在分支入口，因此不能推断“整条残差路径的向量始终 RMS 为 1”。

3.9 和 Q/K 归一化是什么关系？方法与位置要分开看

RMSNorm 是一种具体的归一化算法；Q/K Normalization 描述的是对 Q、K 做归一化这个设计。

因此，Q/K 归一化可以选择 RMSNorm 来实现，也可以采用其他方法。

场景	处理的向量	常见归一化范围	主要作用
Attention 前的 RMSNorm	Hidden State	当前 Token 的整个隐藏维度	调整进入 Attention 分支的特征尺度
FFN 前的 RMSNorm	残差相加后的 Hidden State	当前 Token 的整个隐藏维度	调整进入 FFN 分支的特征尺度
用 RMSNorm 做 Q/K Norm	投影后的 Query、Key	可按每个 Head 的 head_dim 归一化，具体看模型	更直接地控制 Q/K 的尺度

两者可以同时出现：

```
Hidden State
     ↓
RMSNorm（归一化隐藏特征）
     ↓
Wq / Wk / Wv 线性投影
     ↓
Q / K / V
     ↓
可选的 Q/K Norm（也可以使用 RMSNorm）
     ↓
RoPE、Attention 等后续计算
```

为什么前面有了 RMSNorm，后面仍可能需要 Q/K Norm?

因为中间的投影矩阵还可以改变尺度。例如，一个投影矩阵把各分量都放大 100 倍，即使输入已经归一化，投影后的 Q 仍会被放大：

$$W_Q = 100I \quad \Longrightarrow \quad Q = \mathrm{RMSNorm}(X)W_Q = 100\,\mathrm{RMSNorm}(X)$$

这里假设投影前后维度相同，只是为了展示尺度变化。**隐藏状态上的 RMSNorm 并不能代替所有针对 Q/K 的尺度控制。** 是否同时使用两者，取决于模型设计。

3.10 epsilon、零向量与“为什么更简单”

epsilon 放在根号里面

本章采用的分母是：

$$\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}$$

不要误写成先开平方，再加 ϵ；两者在接近零时行为不同。

如果输入全为 0，没有稳定项就会遇到 0 除以 0。有了正的 ϵ：

$$\frac{0}{\sqrt{0+\epsilon}} = 0$$

所以不带偏置的 RMSNorm 对零向量仍输出零向量。

保留 epsilon 后，RMS 也不一定严格等于 1

设平方均值为 m，暂不乘 gamma：

$$m = \frac{1}{d}\sum_i x_i^2, \qquad z_i = \frac{x_i}{\sqrt{m+\epsilon}}$$

那么：

$$\mathrm{RMS}(z) = \sqrt{\frac{m}{m+\epsilon}}$$

只有当 m 远大于 ϵ 时，这个值才接近 1；全零输入的结果仍然是 0。

为什么 RMSNorm 的计算更简单?

与 LayerNorm 相比，它不需要计算均值、逐项减均值，再计算中心化后的方差，而是直接使用平方均值。原论文在多个实验设置中展示了其效率与效果；但具体速度收益会受到硬件、算子融合和张量形状影响，不能把它理解为所有模型上都有固定的加速比例。参见 RMSNorm 原论文。

两者的计算量都随归一化维度数线性增长，RMSNorm 的优势主要是运算和实现更简洁，而不是把复杂度从一个数量级降到另一个数量级。

此外，不减均值并不意味着 RMSNorm 在所有任务上都优于 LayerNorm，也不能不经验证就替换一个已经训练好的模型中的归一化层。

实现时为什么经常临时转成 float32?

平方和求平均会累积数值误差；FP16 中的较大数在平方时还可能溢出。一种常见实现方式是先转成 float32 计算归一化，再转回模型使用的类型。Meta 的上述参考实现就采用了这种处理。

ϵ 的具体取值应与模型配置和实现匹配；下面示例显式使用 10^{-6}，便于复现。

3.11 用代码复现手算，再检查归一化维度

先用 NumPy 写一个只负责前向计算的教学版本。代码中的每一步都对应前面的公式：

```python
import numpy as np


def rms_norm(x, gamma, eps=1e-6):
    x = np.asarray(x, dtype=np.float32)
    gamma = np.asarray(gamma, dtype=np.float32)
    # x: [B, T, D], gamma: [D]
    mean_square = np.mean(x ** 2, axis=-1, keepdims=True)
    # mean_square: [B, T, 1], 每个 Token 一个数
    denominator = np.sqrt(mean_square + eps)
    normalized = x / denominator
    return normalized * gamma


# B=1、T=2、D=4：两个 Token 分别归一化
x = np.array([
    [
        [1.0, 2.0, 2.0, 1.0],
        [10.0, 20.0, 20.0, 10.0],
    ]
], dtype=np.float32)

gamma = np.ones(4, dtype=np.float32)
y = rms_norm(x, gamma)

np.set_printoptions(precision=4, suppress=True)
print("输出形状：", y.shape)
print("输出：")
print(y)
print("每个 Token 的输出 RMS：")
print(np.sqrt(np.mean(y ** 2, axis=-1)))

gamma_adjusted = np.array([1.0, 2.0, 0.5, 1.0], dtype=np.float32)
print("修改 gamma 后，第一个 Token 的输出：")
print(rms_norm(x, gamma_adjusted)[0, 0])
```

输出约为：

```
输出形状： (1, 2, 4)
输出：
[[[0.6325 1.2649 1.2649 0.6325]
  [0.6325 1.2649 1.2649 0.6325]]]
每个 Token 的输出 RMS：
[[1. 1.]]
修改 gamma 后，第一个 Token 的输出：
[0.6325 2.5298 0.6325 0.6325]
```

这里打印的 1 是显示精度下的近似值，实际仍包含 ϵ 的影响。

代码里最关键的是：

- `axis=-1`：沿每个 Token 的特征维度计算。
- `keepdims=True`：保留长度为 1 的最后一维，让分母能正确广播回每个特征。
- `gamma` 的形状为 `[D]`：每一维一个参数，在 batch 和 Token 维度共享。
- NumPy 版本只演示前向运算，gamma 不会自动学习；训练时需要将它注册为框架中的可学习参数。

在支持 `torch.nn.RMSNorm` 的 PyTorch 环境中，可以直接使用内置层：

```
import torch

x = torch.tensor([[
    [1.0, 2.0, 2.0, 1.0],
    [10.0, 20.0, 20.0, 10.0],
]])

norm = torch.nn.RMSNorm(normalized_shape=4, eps=1e-6)
y = norm(x)
print(y.detach())
print(norm.weight.shape)  # torch.Size([4])，对应 gamma
```

这里 `normalized_shape=4` 表示对最后一个轴上的 4 个特征归一化，内置层会管理可学习权重。接口也支持多维 `normalized_shape`；本章使用的是 LLM 中常见的"只归一化最后一个特征轴"的设置。

3.12 三道小练习：确认自己真正理解了

练习 1： 忽略 ϵ，gamma 全为 1，对下面向量做 RMSNorm：

$$x = [2, 2, -2, -2]$$

答案：

$$\mathrm{RMS}(x) = 2, \qquad y = [1, 1, -1, -1]$$

输出均值为 0，是因为这个输入本来就均值为 0；不能据此认为 RMSNorm 总会让均值变成 0。

练习 2： 输入形状为 `[2,3,4]`，归一化最后一维，需要几个分母、几个 gamma 参数？

答案：需要 2×3=6 个分母，分母张量形状为 `[2,3,1]`；gamma 有 4 个参数，输出形状仍为 `[2,3,4]`。

练习 3： 对非零常量向量 $[5, 5, 5, 5]$，RMSNorm 和 LayerNorm 会分别输出什么？

忽略 ϵ 且 gamma 全为 1 时，RMSNorm 输出：

$$[1, 1, 1, 1]$$

LayerNorm 减均值后得到零向量，此时必须保留正的 ϵ。当 beta 为 0 时，输出：

$$[0, 0, 0, 0]$$

这个例子说明：RMSNorm 不会像减均值那样消除所有维度共有的常量分量。这里也要注意，LayerNorm 对常量向量的方差为 0，不能使用“忽略稳定项后的单位方差”说法。

3.13 这一章最值得记住的四点

1. **怎么算：** 平方 → 求平均 → 加 ϵ → 开平方 → 用输入除以它 → 逐维乘 gamma。
2. **对谁算：** 常见 LLM 设置中，每个 Token 沿自己的隐藏维度单独算，不跨 Token 或 batch 混合统计。
3. **和谁不同：** 它不减均值；它控制 RMS，而 L2 归一化控制向量长度。
4. **放在哪里：** 常用于 Attention / FFN 前，也可以作为 Q/K 归一化采用的具体算法。

> **RMSNorm 先按均方根调整一个向量的整体尺度，再让模型通过可学习参数调整各维幅度。**

---

4. GQA：Grouped Query Attention

4.1 从 MHA 开始

标准 Multi-Head Attention（MHA）中，每个 Head 都有自己的：

$$Q_i, K_i, V_i$$

例如：

$$H = 8$$

那么：

```
Q Head：8 个
K Head：8 个
V Head：8 个
```

可以表示成：

```
Q1 -> K1 V1
Q2 -> K2 V2
Q3 -> K3 V3
Q4 -> K4 V4
Q5 -> K5 V5
Q6 -> K6 V6
Q7 -> K7 V7
Q8 -> K8 V8
```

4.2 MHA 的问题

训练时问题不一定特别明显。

但是在 LLM 自回归推理中，历史 Token 的 K 和 V 需要存进：

$$KV\ Cache$$

如果每个 Attention Head 都保存一份 K/V：

```
32 个 Q Head
32 个 K Head
32 个 V Head
```

那么 KV Cache 会非常大。

因此一个自然的问题是：

> Query 一定需要和 K/V 一一对应吗？

答案是不一定。

4.3 MQA：极端做法

Multi-Query Attention（MQA）直接让所有 Q Head 共用一套 K/V：

```
Q1 ┐
Q2 ┤
Q3 ┤
Q4 ┼──> K1 / V1
Q5 ┤
Q6 ┤
Q7 ┤
Q8 ┘
```

即：

```
Query Head = 8
KV Head    = 1
```

优点：

- KV Cache 极小
- 推理非常省显存

缺点：

- 所有 Query Head 共用同一个 K/V

- 表达能力可能下降

4.4 GQA 是 MHA 和 MQA 的折中

Grouped Query Attention：

> 将多个 Query Head 分成若干组，每组共享一套 K/V。

例如：

$$N_Q = 8$$

$$N_{KV} = 2$$

那么每：

$$\frac{8}{2} = 4$$

个 Query Head 共用一个 KV Head。

```
Q1 ┐
Q2 ┤
Q3 ┤──> K1 / V1
Q4 ┘

Q5 ┐
Q6 ┤
Q7 ┤──> K2 / V2
Q8 ┘
```

4.5 MHA / GQA / MQA 对比

假设：

$$N_Q = 32$$

则：

方法	Q Head	KV Head
MHA	32	32
GQA	32	8
MQA	32	1

所以：

```
MHA ———— GQA ———— MQA
表达能力高                    KV Cache 小
KV Cache 大                   共享程度高
```

GQA 位于中间。

4.6 GQA 为什么能降低 KV Cache?

KV Cache 大小与:

$$N_{KV}$$

直接相关，而不是只与 Query Head 数量相关。

假设:

- Layer = 32
- Sequence Length = 4096
- KV Head = 32
- Head Dim = 128
- FP16 = 2 Bytes

每层 K Cache:

$$4096 \times 32 \times 128 \times 2$$

V Cache 同样大。

总 KV Cache:

$$2 \times 32 \times 4096 \times 32 \times 128 \times 2$$

约为:

$$2\ GB$$

如果使用 GQA:

$$N_{KV} = 8$$

KV Cache 大约变成:

$$\frac{8}{32} = \frac{1}{4}$$

即约:

$$0.5\ GB$$

4.7 GQA 中 Q 如何找到对应的 KV Head?

例如:

```
num_attention_heads = 32
num_key_value_heads = 8
```

则:

$$n_{\text{rep}} = \frac{32}{8} = 4$$

一个 KV Head 对应四个 Query Head。

代码中经常看到:

```
n_rep = num_attention_heads // num_key_value_heads
```

然后把 K/V 在 Head 维度重复:

```
K = repeat_kv(K, n_rep)
V = repeat_kv(V, n_rep)
```

注意:

> 逻辑上是多个 Q Head 共用同一个 K/V,
> 某些代码实现为了方便矩阵计算,会将 K/V 做 view / expand / repeat。

4.8 一句话总结

> **GQA 通过让多个 Query Head 共享 K/V Head,在尽量保留 MHA 表达能力的同时,大幅降低 KV Cache 和推理开销。**

---

5. RoPE: Rotary Position Embedding

5.1 为什么 Transformer 需要位置编码?

Attention 本身只看:

$$QK^T$$

如果没有位置编码：

```
我 喜欢 猫
猫 喜欢 我
```

模型很难仅靠 Attention 区分 Token 所在位置。

因此必须引入 Position Information。

5.2 RoPE 的基本思想

RoPE：

> Rotary Position Embedding，旋转位置编码。

核心思想：

> 根据 Token 的位置，把 Q 和 K 在二维平面中旋转一定角度。

假设原向量：

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

二维旋转矩阵为：

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

旋转以后：

$$x' = R(\theta)x$$

5.3 一个简单例子

假设：

$$q = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Token 位于 position = 1。

如果对应旋转角：

$$\theta = 90^\circ$$

那么：

$$R(90^\circ) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

于是：

$$q' = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

也就是说：

```
原始 Q：
----->

经过 RoPE：
  ↑
  |
```

位置不同，旋转角不同。

5.4 高维向量如何旋转？

实际 Q/K 不可能只有 2 维。

例如：

$$q = [q_0, q_1, q_2, q_3, q_4, q_5, \dots]$$

RoPE 将维度两两分组：

$$(q_0, q_1)$$

$$(q_2, q_3)$$

$$(q_4, q_5)$$

每一对按照不同频率进行旋转。

例如：

```
维度 0,1  -> θ0
维度 2,3  -> θ1
维度 4,5  -> θ2
...
```

5.5 不同维度为什么使用不同频率？

经典 RoPE 中：

$$\theta_i = 10000^{-\frac{2i}{d}}$$

Token 位于位置 m 时：

$$m\theta_i$$

作为旋转角。

于是：

$$R(m\theta_i)$$

不同维度具有不同的位置变化速度：

```
高频维度：对位置变化敏感
低频维度：适合表示更长距离的位置变化
```

这与正弦位置编码的多频率设计思想类似。

5.6 RoPE 最重要的性质：相对位置

假设位置：

$$m$$

上的 Query：

$$R(m\theta)q$$

位置：

$$n$$

上的 Key：

$$R(n\theta)k$$

它们点积：

$$(R(m\theta)q)^T(R(n\theta)k)$$

根据旋转矩阵性质：

$$R(m\theta)^T R(n\theta) = R((n-m)\theta)$$

因此：

$$(R(m\theta)q)^T(R(n\theta)k) = q^T R((n-m)\theta)k$$

注意右边只与：

$$n - m$$

有关。

也就是说：

> Attention 最终可以自然感知两个 Token 的相对距离。

5.7 一个语言例子

句子：

```
小明 昨天 去了 北京
```

假设：

```
“小明”位置 = 0
“北京”位置 = 4
```

那么：

$$\Delta p = 4$$

换一个句子：

```
今天 小明 昨天 去了 北京
```

现在：

```
“小明”位置 = 1
“北京”位置 = 5
```

仍然：

$$\Delta p = 4$$

RoPE 能够较自然地保留这种相对位置关系。

5.8 RoPE 为什么只作用于 Q/K?

因为 Attention Score 为：

$$QK^T$$

位置关系真正参与计算的地方就在 Q 与 K 的点积。

所以通常：

```
q = apply_rotary_pos_emb(q)
k = apply_rotary_pos_emb(k)
```

而：

```
v
```

不需要进行 RoPE。

5.9 一句话总结

> **RoPE 通过对不同位置的 Q/K 施加不同角度的旋转，让 Attention 点积天然包含相对位置信息。**

相对位置的数学性质并不自动保证无限长度的泛化。下一章继续讨论 YaRN 如何调整 RoPE，以帮助模型适应更长上下文。

---

6. YaRN：让 RoPE 更稳地适应长上下文

> 学习目标：解释为什么 RoPE 不能仅靠“把位置编号变大”就可靠处理长文本，理解 YaRN 的分频段调整与注意力尺度调整，并能算出一个小例子。
>
> 本章接着第 5 章的 RoPE。YaRN 是针对 RoPE 的上下文扩展方法，其两项核心设计可对照 YaRN 原论文 第 3 节。

6.1 为什么能算出位置 10000，不代表模型就能读懂 10000 个 Token?

假设一个模型原先主要在最长 4096 个 Token 的上下文上训练，我们现在希望它处理 16384 个 Token。

从程序角度看，把 RoPE 的位置编号继续设为 4096、4097……并不困难。正弦、余弦对这些位置仍然可以计算。

困难在于：**公式能计算，不等于模型已经学会如何利用这些位置关系。**

回顾 RoPE，第 i 对特征在位置 m 上的旋转角是：

$$\phi_i(m) = m\theta_i$$

其中 θ_i 是每前进一个 Token 增加的角度，单位是弧度。位置越远，累计角度越大。

训练长度内，有些频率已经转过很多圈，有些频率连一圈都没有转完。超出训练长度后，各频率组合形成的位置模式、模型遇到的相对距离，以及要关注的 Token 数量都发生变化。

因此，不能把 RoPE 的"点积包含相对位置"理解为"可以不受训练分布影响地无限外推"。YaRN 尝试调整这套位置表示，让它更适合目标长度；它不直接压缩文本内容。

6.2 最简单的办法：把位置等比例压缩

设：

$$L_0 = 4096, \qquad L_1 = 16384, \qquad s = \frac{L_1}{L_0} = 4$$

这里 L_0 是原始参考上下文长度，L_1 是目标长度，s 是扩展倍数。

一种简单思路叫 **Position Interpolation，位置插值**：把用于计算旋转角的位置除以 s。

$$m' = \frac{m}{s}$$

例如，新文本中位置为 12000 的 Token，在计算 RoPE 时使用：

$$m' = \frac{12000}{4} = 3000$$

这里没有删除 Token，也没有把每 4 个 Token 合并成一个。只是位置坐标的刻度缩小了。

实际位置 m	用于插值的位置 $m/4$
0	0
1	0.25
4	1
4096	1024
12000	3000

非整数坐标没有问题，因为它只用于计算旋转角。

把位置除以 s，与把所有频率除以 s，在角度上等价：

$$\frac{m}{s}\theta_i = m\frac{\theta_i}{s}$$

因此实现时通常可以保留位置编号，改用较小的频率。

代价是什么？相邻 Token 的角度差也缩小了。 原来相隔 1 个 Token 的角度差是 θ_i，现在变成 $\theta_i/4$。高频中原本较精细的局部位置变化也被一起压缩，可能影响模型已经学到的短距离模式。

这不表示模型彻底分不清相邻 Token，而是说位置表示的尺度变化需要适应。

6.3 YaRN 的关键直觉：不同频率不必采用同一种压缩程度

可以把 RoPE 的不同频率想象成转速不同的表针：

- 转得快的表针，相邻位置就能产生明显变化。
- 转得慢的表针，要跨越更远的距离才变化明显。

YaRN 中的分频段设计，倾向于保留高频的局部变化，同时让低频适应更长的位置范围。中间频段采用平滑过渡。

如何判断某一对维度属于哪种情况？先计算它转一整圈需要多少 Token：

$$\lambda_i = \frac{2\pi}{\theta_i}$$

这个量叫波长。再看它在原始参考长度内大约能转多少圈：

$$r_i = \frac{L_0}{\lambda_i} = \frac{L_0\theta_i}{2\pi}$$

这里把长度当作连续跨度来估计圈数；实际位置通常从 0 编号。

原始窗口内的圈数	直观含义	调整思路
很多圈	高频，较短的波长	尽量保留原频率
很少，甚至不足一圈	低频，较长的波长	更充分地压缩频率
介于两者之间	过渡频段	混合原频率和压缩后的频率

注意两种“维度”的说法：这里的高频、低频指旋转频率的大小，不是向量维度总数的大小。标准 RoPE 中，随着频率对的索引 i 增大，频率反而通常变小。

6.4 把直觉写成公式：有些保留，有些压缩

为了便于手算，本节采用原论文按圈数构造过渡权重的表达。设两个阈值：

$$\alpha = 1, \qquad \beta = 32$$

它们分别表示较慢与较快频段的边界。实际模型可以使用不同配置；这些值不是数学定律。

定义“保留原频率的比例”：

$$g_i = \begin{cases} 0, & r_i \le \alpha \\ \dfrac{r_i - \alpha}{\beta - \alpha}, & \alpha < r_i < \beta \\ 1, & r_i \ge \beta \end{cases}$$

这里使用 g_i 表示频段混合权重，避免与 RMSNorm 中的可学习 gamma 混淆。给定长度、频率和阈值后，g_i 就由公式确定。

调整后的频率是：

$$\boxed{\theta'_i = g_i\theta_i + (1-g_i)\frac{\theta_i}{s}}$$

逐个看，就很容易理解：

- $g_i = 1$：完全使用原频率，$\theta'_i = \theta_i$。
- $g_i = 0$：完全使用插值频率，$\theta'_i = \theta_i/s$。
- $g_i = 0.5$：两种频率各占一半。

只要 $s \geq 1$，调整结果就满足：

$$\frac{\theta_i}{s} \leq \theta'_i \leq \theta_i$$

这一步是 YaRN 的分频段频率调整部分，完整 YaRN 还包含第 6.6 节的注意力尺度调整。

实现细节： 常见 Transformers 实现先把圈数阈值换算成维度索引，再在索引上构造线性过渡，边界还可能取整。因此，其过渡带的具体数值不一定等于本节直接对圈数做线性插值的结果。下面的手算与代码统一采用本节公式；复现模型时应遵循该模型的实现。可对照 Transformers 的 YaRN 实现。

6.5 一个完整的频率例子：4096 扩展到 16384

为了让表格足够小，假设参与 RoPE 的维度数只有 8，也就是 4 对特征。使用基数 10000：

$$\theta_i = 10000^{-2i/8}, \qquad i = 0, 1, 2, 3$$

得到：

$$\theta = [1,\ 0.1,\ 0.01,\ 0.001]$$

设 $L_0 = 4096$、$s = 4$，并使用上一节的阈值。

特征对 i	原频率 θ_i	原窗口内圈数 r_i	保留比例 g_i	新频率 θ'_i
0	1	651.8986	1	1
1	0.1	65.1899	1	0.1
2	0.01	6.5190	0.1780	0.00383524
3	0.001	0.6519	0	0.00025

我们把第三行单独算一遍：

$$r_2 = \frac{4096 \times 0.01}{2\pi} \approx 6.5190$$

$$g_2 = \frac{6.5190 - 1}{32 - 1} \approx 0.1780$$

$$\theta'_2 = 0.1780 \times 0.01 + (1 - 0.1780) \times \frac{0.01}{4} \approx 0.00383524$$

再与统一位置插值比较：

$$\theta_{\text{PI}} = [0.25,\ 0.025,\ 0.0025,\ 0.00025]$$

统一插值把四种频率全部除以 4；本例的 YaRN 频率调整保留了前两种高频，第三种部分压缩，最后一种完全压缩。

假设两个 Token 相隔 1000 个位置，在最低频那一对特征上：

$$\Delta\phi_{\text{original}} = 1000 \times 0.001 = 1$$

$$\Delta\phi_{\text{new}} = 1000 \times 0.00025 = 0.25$$

而最高频仍使用原来的每步旋转速度。这样就能看出"为长距离留出范围，同时尽量保留局部变化"的含义。

6.6 还有一步：调整注意力的温度

前面改变的是 Q/K 的旋转频率。YaRN 还引入注意力尺度调整，用于适应长上下文中的注意力分布。

把普通的注意力分数记为：

$$z_j = \frac{q^T k_j}{\sqrt{d_h}}$$

温度形式是：

$$p_j = \text{softmax}(z/\tau)_j$$

- $\tau > 1$：分数差被缩小，概率通常更平缓。
- $0 < \tau < 1$：分数差被放大，概率通常更集中。

更长的上下文中会出现更多候选 Key。对于同一个 Query，注意力分布可能变得更分散。温度缩放提供了一个调整这种分布的办法，但最佳幅度仍然需要实验确定。

原论文针对 LLaMA / Llama 2 给出的一个经验设置，可以写成 Q/K 的幅度因子：

$$a(s) = 1 + 0.1 \ln s, \qquad s \geq 1$$

同时将旋转后的 Q 和 K 乘以 a：

$$q' = aR_{\theta'}(m)q, \qquad k' = aR_{\theta'}(n)k$$

此时它们的点积会乘以 **平方**：

$$\frac{(q')^T k'}{\sqrt{d_h}} = a^2 \frac{(R_{\theta'}(m)q)^T (R_{\theta'}(n)k)}{\sqrt{d_h}}$$

对应的温度就是：

$$\tau = \frac{1}{a^2}$$

不要把"Q/K 的幅度乘 a"和"注意力分数乘 a"混成一件事。 两边都乘 a 时，点积乘的是 a^2。

对于 $s = 4$：

$$a \approx 1.1386, \qquad a^2 \approx 1.2965, \qquad \tau \approx 0.7713$$

假设已经完成频率调整后的三个分数是 $[1, 2, 3]$，只比较这一额外幅度调整：

情况	Softmax 概率
未加幅度调整	$[0.0900, 0.2447, 0.6652]$
分数乘约 1.2965 后	$[0.0555, 0.2028, 0.7417]$

较大分数对应的概率变高了。

工程上可以把幅度因子合并进 RoPE 的 cos/sin，也可以在等价的注意力缩放处实现。**这两个位置不要重复乘。**若模型仅对部分维度使用 RoPE，或另有注意力缩放设计，也不能直接把本节"所有 Q/K 维度统一缩放"的推导照搬过去。

6.7 和 RoPE、KV Cache、长上下文训练怎样配合？

可以把静态 YaRN 的使用过程分为三个阶段：

```
先确定原始长度 L0、目标长度 L1、扩展倍数 s
                    ↓
按频段得到新频率，并确定注意力幅度因子
                    ↓
按模型方案进行长上下文适应，并评估长、短文本效果
                    ↓
推理时使用相同的位置编码配置
                    ↓
按新配置计算 Q/K，写入和读取 KV Cache
```

这里"静态"表示一次生成过程中使用固定的扩展配置。YaRN 本身也存在结合动态缩放的使用方式，但动态改变频率时需要额外处理缓存一致性。

例如，假设缓存中的旧 Key 使用频率 θ_{old}，新 Query 却使用 θ_{new}，它们的旋转项会变成：

$$R(m\theta_{\text{new}})^T R(n\theta_{\text{old}}) = R(n\theta_{\text{old}} - m\theta_{\text{new}})$$

这一般不能化成统一频率下的相对位置项：

$$R((n-m)\theta')$$

所以不能在保留旧缓存的同时随意修改频率配置。动态方法需要按其实现维护一致的旋转方式，例如保存旋转前的 Key 并重新应用位置变换。

还要区分三个不同问题：

问题	主要相关机制
模型如何编码更长的位置范围	RoPE 扩展方法，例如 YaRN
模型是否真正学会利用远处的信息	长上下文适应、训练数据与评估
更长序列的计算和显存如何承担	Attention 实现、GQA、KV Cache 管理等

YaRN 不会自动缩小 KV Cache。 在其他条件不变、缓存所有历史 K/V 的情况下，上下文从 4096 增加到 16384，KV Cache 约变为 4 倍。全注意力的 Token 两两交互部分，在完整 Prefill 时计算量约变为 16 倍；这不是对整个推理过程耗时的固定倍率预测。

6.8 用 NumPy 复现频率表和温度例子

下面实现的是第 6.4 节的论文公式教学版，不是某个库的逐位数值复刻：

```
import numpy as np


def yarn_frequency_demo(
    rotary_dim=8,
    base=10000.0,
    original_length=4096,
    factor=4.0,
    alpha=1.0,
    beta=32.0,
):
    assert rotary_dim > 0 and rotary_dim % 2 == 0
    assert base > 1 and original_length > 0
    assert factor >= 1 and 0 < alpha < beta
    i = np.arange(rotary_dim // 2, dtype=np.float64)
    theta = base ** (-2 * i / rotary_dim)
    rotations = original_length * theta / (2 * np.pi)
    keep = np.clip((rotations - alpha) / (beta - alpha), 0, 1)
    new_theta = keep * theta + (1 - keep) * theta / factor
    amplitude = 1 + 0.1 * np.log(factor)
    return theta, rotations, keep, new_theta, amplitude


def softmax(values):
    shifted = values - np.max(values)
    exp_values = np.exp(shifted)
    return exp_values / exp_values.sum()


theta, rotations, keep, new_theta, amplitude = yarn_frequency_demo()
np.set_printoptions(precision=6, suppress=True)
print("原频率：", theta)
print("圈数：", rotations)
print("保留比例：", keep)
print("新频率：", new_theta)
print("Q/K 幅度因子：", round(amplitude, 6))
scores = np.array([1.0, 2.0, 3.0])
print("原概率：", softmax(scores))
print("调整后概率：", softmax(amplitude ** 2 * scores))
```

关键结果约为：

```
原频率： [1.    0.1   0.01  0.001]
圈数： [651.898647 65.189865 6.518986 0.651899]
保留比例： [1.       1.       0.178032 0.      ]
新频率： [1.       0.1      0.003835 0.00025 ]
Q/K 幅度因子： 1.138629
原概率： [0.090031 0.244728 0.665241]
调整后概率： [0.055477 0.202844 0.741679]
```

可以把 factor 改成 1 再观察：频率会回到原值，幅度因子也回到 1。这是一个很直观的边界检查。

6.9 常见误解与练习

误解 1：把最大长度配置改大，就一定扩展成功。

配置允许输入更长序列，只说明程序能够接收它。模型是否能正确提取远处的信息，还要通过长文问答、检索、困惑度等评估，同时检查短文本表现。

误解 2：所有频率都应该除以扩展倍数。

那是统一位置插值的做法。YaRN 的分频段部分让不同频率承担不同程度的压缩。

误解 3：YaRN 就是给 Q/K 做 RMSNorm。

RMSNorm 根据当前输入向量计算分母；YaRN 的频率与幅度调整由位置编码方案和扩展配置决定。二者作用不同，也可能同时出现在模型中。

练习 1： 原始长度 8192、目标长度 32768，扩展倍数是多少？统一位置插值下，位置 20000 对应哪个坐标？

答案：s=4，对应 5000。

练习 2： 原频率为 0.02，s=4，保留比例 g=0.5，新频率是多少？

$$\theta' = 0.5 \times 0.02 + 0.5 \times \frac{0.02}{4} = 0.0125$$

练习 3： Q 和 K 都乘以 1.2，点积乘多少？

答案：乘以 $1.2^2 = 1.44$。

6.10 本章小结

> **YaRN 在 RoPE 的基础上，让不同频段采用不同程度的缩放，并调整注意力尺度，以帮助模型适应更长上下文。能否稳定利用长文本，还需要相应的模型适应与验证。**

下一章的 KV Cache 解决的是历史 K/V 如何复用；两者一起使用时，位置编码配置必须与缓存保持一致。

---

7. KV Cache

7.1 为什么需要 KV Cache?

LLM 是自回归生成:

$$P(x_t|x_1, x_2, \ldots, x_{t-1})$$

例如 Prompt:

```
我 喜欢 学习
```

模型需要生成:

```
人工
```

然后:

```
智能
```

完整过程:

```
我 喜欢 学习
        ↓
       人工

我 喜欢 学习 人工
             ↓
            智能
```

7.2 没有 KV Cache 会发生什么?

第 1 步:

```
我 喜欢 学习
```

计算:

```
Q1 K1 V1
Q2 K2 V2
```

```
Q3 K3 V3
```

生成：

```
人工
```

第 2 步：

```
我 喜欢 学习 人工
```

如果没有 Cache，需要重新计算：

```
我       -> K1 V1
喜欢     -> K2 V2
学习     -> K3 V3
人工     -> K4 V4
```

第 3 步：

又需要重新计算：

```
K1 V1
K2 V2
K3 V3
K4 V4
K5 V5
```

历史 Token 的 K/V 被反复计算。

7.3 KV Cache 的思想

既然旧 Token 的：

$$K, V$$

已经算过，而且模型参数在推理过程中不变，那么可以直接保存。

例如 Prompt 阶段计算：

```
Token    K       V
我       K1      V1
```

```
喜欢      K2      V2
学习      K3      V3
```

保存：

```
KV Cache:
[K1,K2,K3]
[V1,V2,V3]
```

生成“人工”时，只计算新 Token：

```
K4,V4
```

然后追加：

```
K Cache = [K1,K2,K3,K4]
V Cache = [V1,V2,V3,V4]
```

7.4 Prefill 与 Decode

LLM 推理通常分成两个阶段。

Prefill

输入完整 Prompt：

```
我喜欢学习人工智能
```

一次性并行计算所有 Token 的：

$$Q, K, V$$

并建立 KV Cache。

特点：

- Token 多
- 矩阵计算规模大
- GPU Compute Intensive

Decode

每次只生成一个或少量新 Token：

```
Step 1 -> Token 1
Step 2 -> Token 2
Step 3 -> Token 3
```

每一步：

- 只计算新 Token 的 Q/K/V
- 读取历史 KV Cache
- 新 K/V 加入 Cache

特点：

- 每次计算量不大
- 但需要反复读取大量 KV Cache
- 更容易受显存带宽影响

7.5 Decode 中 Attention 是如何计算的?

假设已经有：

$$K_1, K_2, K_3, K_4$$

生成第 5 个 Token 时，只计算：

$$Q_5$$

然后：

$$Q_5 \begin{bmatrix} K_1 \\ K_2 \\ K_3 \\ K_4 \end{bmatrix}^T$$

得到：

$$[a_1, a_2, a_3, a_4]$$

Softmax：

$$[\alpha_1, \alpha_2, \alpha_3, \alpha_4]$$

最后：

$$O_5 = \alpha_1 V_1 + \alpha_2 V_2 + \alpha_3 V_3 + \alpha_4 V_4$$

因此历史 Token 不需要重新做 K/V 投影。

7.6 KV Cache 显存公式

常用估算：

$$M_{KV} = 2 \times L \times T \times N_{KV} \times D_h \times B$$

其中：

- 2：K + V
- L：Transformer 层数
- T：Sequence Length
- N_{KV}：KV Head 数
- D_h：Head Dimension
- B：每个元素的字节数

如果有 Batch Size：

$$M_{KV} = 2 \times B_{\text{batch}} \times L \times T \times N_{KV} \times D_h \times B$$

7.7 具体例子

假设：

```
Layer            = 32
Sequence Length  = 4096
KV Head          = 8
Head Dim         = 128
dtype            = FP16 = 2 Bytes
Batch Size       = 1
```

那么：

$$M = 2 \times 32 \times 4096 \times 8 \times 128 \times 2$$

计算结果：

$$M = 536,870,912\ Bytes$$

约：

$$512\ MB$$

仅一个请求就需要约 512 MB KV Cache。

如果：

$$Batch = 16$$

则：

$$16 \times 512\ MB = 8\ GB$$

因此长上下文推理中：

> **KV Cache 往往比模型计算本身更容易成为显存瓶颈。**

7.8 GQA 为什么和 KV Cache 强相关?

因为：

$$M_{KV} \propto N_{KV}$$

MHA：

$$N_{KV} = 32$$

GQA：

$$N_{KV} = 8$$

则：

$$M_{\text{GQA}} = \frac{8}{32} M_{\text{MHA}} = \frac{1}{4} M_{\text{MHA}}$$

这也是现代 LLM 广泛采用 GQA 的重要原因。

7.9 KV Cache 不缓存 Q 吗?

通常不缓存 Q。

原因：

对于第 t 个 Token，只需要：

$$Q_t$$

去查询历史：

$$K_1, \ldots, K_t$$

下一步生成 Token $t+1$ 时，需要的是：

$$Q_{t+1}$$

旧的：

$$Q_t$$

不会再使用。

而历史 K/V 会被后续每一步不断使用。

因此只缓存：

$$K, V$$

而不是：

$$Q, K, V$$

7.10 一句话总结

> **KV Cache 用显存换计算：缓存历史 Token 的 K/V，避免每次 Decode 都重新计算整个历史序列。**

---

8. RoPE 和 KV Cache 是怎么一起工作的?

以下先说明固定 RoPE 配置下的缓存流程。如果使用静态 YaRN，也要用同一套频率和幅度配置生成当前 Q 与缓存 K；动态改变配置时的注意事项见第 6.7 节。

这是理解现代 LLM 推理非常关键的一点。

假设现在已经有：

```
Token 0
Token 1
Token 2
Token 3
```

它们对应位置：

$$0, 1, 2, 3$$

计算 K 时先做 RoPE：

$$K_i' = R(i\theta)K_i$$

然后缓存的实际上通常是已经带位置信息的：

$$K_0', K_1', K_2', K_3'$$

生成第 4 个 Token 时：

$$Q_4' = R(4\theta)Q_4$$

然后直接：

$$Q_4'[K_0', K_1', K_2', K_3']^T$$

因此：

> KV Cache 中缓存的 K 必须与其原始 Token Position 正确对应。

如果 Position 错了：

$$R(i\theta)$$

就错了，Attention 的相对位置信息也会错误。

---

9. RMSNorm + Q/K Normalization + RoPE + GQA + KV Cache 的 Attention 流程

以下仍以使用 Pre-Norm 的模型为例。假设输入 Hidden State 为 X，先在 Attention 分支入口归一化：

$$\widetilde{X} = \mathrm{RMSNorm}(X)$$

再对归一化后的特征做线性投影：

$$Q = \widetilde{X} W_Q$$

$$K = \widetilde{X} W_K$$

$$V = \widetilde{X} W_V$$

如果模型使用 Q/K Normalization：

$$Q = \mathrm{Norm}(Q)$$

$$K = \mathrm{Norm}(K)$$

然后应用选定的位置编码配置。普通 RoPE 如下；若使用 YaRN，则使用其调整后的频率，并按模型约定处理注意力幅度因子。YaRN 是修改这一位置变换的配置，不是在普通 RoPE 后再额外旋转一遍。

$$Q = \mathrm{RoPE}(Q, pos)$$

$$K = \mathrm{RoPE}(K, pos)$$

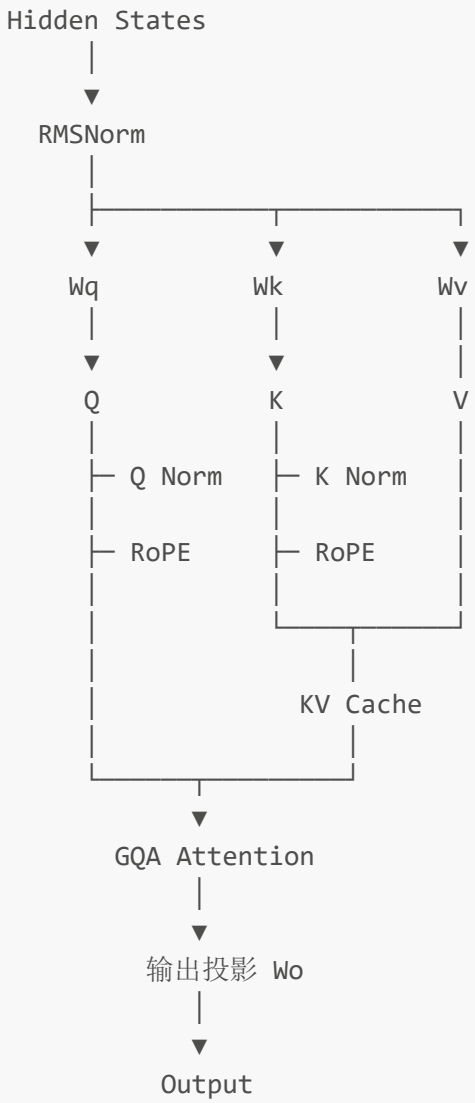
再将 K/V 写入：

$$KV\ Cache$$

如果使用 GQA：

```
多个 Query Head
        ↓
共享少量 KV Head
        ↓
Attention
```

完整流程可以写成：



上图只展开 Attention 分支。其输出还要加回原始 X，得到 H；之后执行 RMSNorm → FFN / MoE → 加回 H。残差连接的完整位置见第 3.8 节。

---

10. FFN：对每个 Token 的特征做非线性加工

> 学习目标：能解释 Attention 之后为什么还需要 FFN，手算一次"升维—激活—降维"，并看懂现代 LLM 中常见的 SwiGLU。
>
> 本章统一使用行向量写法：输入在左、权重在右，和前面的 $Q = XW_Q$ 保持一致。所有教学矩阵都是人为选取，便于手算，不代表真实模型已学到的权重。

10.1 Attention 之后，FFN 还要做什么？

FFN 是 **Feed-Forward Network，前馈神经网络。**在 Transformer 里，通常指每个层中对 Token 特征逐位置处理的那部分网络，也常在代码中叫 MLP。

先用一个工作分工建立直觉：

```
Attention：让当前 Token 从允许关注的其他 Token 获取信息
                                ↓
FFN：根据当前 Token 已经形成的特征，做进一步的非线性变换
```

例如，“苹果发布了新手机”和“苹果吃起来很甜”中的“苹果”，在结合不同上下文后，会形成不同的隐藏状态。FFN 再处理这些已经带有上下文的特征。

这只是一种便于理解的分工。Attention 自身也有可学习变换和非线性计算，不能把它看成只负责机械地搬运信息。

FFN 的一个重要特点是：**同一层内，不同 Token 使用相同的权重，但分别计算各自的输出。** 这种逐位置、共享参数的结构见 Transformer 原论文第 3.3 节。

10.2 最基本的结构：升维、激活、降维

经典的两层 FFN 可以写成：

$$\boxed{\mathrm{FFN}(x) = \phi(xW_1 + b_1)W_2 + b_2}$$

把它拆开：

$$z = xW_1 + b_1$$

$$h = \phi(z)$$

$$y = hW_2 + b_2$$

符号	形状	作用
x	$[D]$	一个 Token 的输入特征
W_1	$[D, M]$	把特征变换到中间维度
b_1	$[M]$	第一层偏置
z、h	$[M]$	中间表示
ϕ	逐元素作用	引入非线性
W_2	$[M, D]$	映射回原来的隐藏维度
b_2	$[D]$	第二层偏置
y	$[D]$	FFN 分支的输出

通常中间维度 M 大于输入维度 D，经典设置里常取 M=4D，但具体倍数是模型设计选择，不是 FFN 的定义要求。

```
一个 Token：D 维
      ↓
第一层线性变换
```

```
        ↓
M 维，通常更宽
        ↓
逐元素激活函数
        ↓
第二层线性变换
        ↓
回到 D 维
```

回到 D 维的一个直接好处是：FFN 输出可以与原来的 D 维隐藏状态做残差相加。

10.3 完整手算：2 维输入，3 维中间层，2 维输出

为了避免大矩阵挡住直觉，取：

$$x = [2, -1], \qquad D = 2, \qquad M = 3$$

省略偏置，使用 ReLU：

$$\mathrm{ReLU}(u) = \max(0, u)$$

设置两个权重矩阵：

$$W_1 = \begin{bmatrix} 1 & -1 & 2 \\ 1 & 1 & -1 \end{bmatrix}, \qquad W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix}$$

第一步：线性组合，得到三个中间特征。

$$z = xW_1 = [2 \times 1 + (-1) \times 1,\ 2 \times (-1) + (-1) \times 1,\ 2 \times 2 + (-1) \times (-1)]$$

$$z = [1, -3, 5]$$

第一层在计算三种组合：

$$z_1 = x_1 + x_2, \qquad z_2 = -x_1 + x_2, \qquad z_3 = 2x_1 - x_2$$

因此，升维并不是简单地在向量后面补零，而是构造更多种可学习的特征组合。

第二步：ReLU 保留正值，把负值置零。

$$h = \mathrm{ReLU}([1, -3, 5]) = [1, 0, 5]$$

这一步使结果依赖于输入落在哪个区域，例如某个线性组合是正还是负。

第三步：重新组合，映射回 2 维。

$$y = hW_2 = [1 \times 1 + 0 \times 0 + 5 \times 1,\ 1 \times 0 + 0 \times 1 + 5 \times (-1)]$$

$$\boxed{y = [6, -5]}$$

完整过程可以压缩成：

```
[2, -1]
   ↓ W1
[1, -3, 5]
   ↓ ReLU
[1, 0, 5]
   ↓ W2
[6, -5]
```

注意：这里算出的是 **FFN 分支输出**，还没有加残差。如果模块最后执行 $x + \mathrm{FFN}(x)$，那么结果才是 $[8, -6]$。

10.4 为什么中间一定要有非线性？

假设去掉激活函数，也忽略偏置：

$$y = (xW_1)W_2 = x(W_1W_2)$$

令：

$$W_{\text{equiv}} = W_1W_2$$

那么两层就可以合并成一层：

$$y = xW_{\text{equiv}}$$

即使中间升到了很高维，只要全部是线性运算，最终依然只是一次线性变换。加入偏置后，多层也仍然可以合并成一次仿射变换。

沿用上一节的矩阵：

$$W_1W_2 = \begin{bmatrix} 3 & -3 \\ 0 & 2 \end{bmatrix}$$

去掉 ReLU 后：

$$y = [2, -1] \begin{bmatrix} 3 & -3 \\ 0 & 2 \end{bmatrix} = [6, -8]$$

有 ReLU 时则是 $[6, -5]$。

还可以检查线性映射应当满足的一个性质：输入反号，输出也反号。对于上一节的带 ReLU 的 FFN：

$$\mathrm{FFN}([2, -1]) = [6, -5]$$

但：

$$\mathrm{FFN}([-2, 1]) = [0, 3]$$

第二个结果并不是 $[-6, 5]$。这说明它已经不再是单一线性变换。

升维提供更多特征组合，非线性让这些组合随输入产生不同响应，降维再把结果组织回隐藏状态。

10.5 为什么说 FFN “逐 Token 计算”？用两个 Token 看一次

仍然使用同一组 W_1、W_2：

$$x_A = [2, -1], \qquad x_B = [1, 1]$$

Token A 的结果已经算过：

$$\mathrm{FFN}(x_A) = [6, -5]$$

Token B 则是：

$$x_B W_1 = [2, 0, 1]$$

$$\mathrm{ReLU}([2, 0, 1]) = [2, 0, 1]$$

$$\mathrm{FFN}(x_B) = [3, -1]$$

把两个 Token 堆成矩阵，只是方便同时计算：

$$X = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix} \quad \Longrightarrow \quad \mathrm{FFN}(X) = \begin{bmatrix} 6 & -5 \\ 3 & -1 \end{bmatrix}$$

第一行的计算不需要读取第二行的输入；第二行也不需要第一行。GPU 可以把这些独立运算合并成大矩阵乘法，并不意味着 FFN 在 Token 之间交换了信息。

对于一般输入：

$$X \in \mathbb{R}^{B \times T \times D}$$

形状变化是：

```
[B, T, D] → [B, T, M] → [B, T, M] → [B, T, D]
```

B 和 T 都不变，变化的是最后的特征轴。

但不要因此说“FFN 完全不知道上下文”：它接收的 Hidden State 通常已经经过 Attention，里面已经包含上下文信息。独立的是本次 FFN 运算，而不是整个模型对各 Token 的处理。

10.6 激活函数为什么有 ReLU、GELU、SiLU 这么多种？

它们都在提供非线性，但响应曲线不同：

函数	公式	直观特点
ReLU	$\max(0, x)$	正值直接通过，负值置零
GELU	$x\Phi(x)$	用标准正态分布的累积分布函数 Φ 平滑调节输入
SiLU / Swish（系数为 1）	$x/(1+e^{-x})$	用 Sigmoid 对输入进行平滑调节

以 SiLU 为例：

$$\mathrm{SiLU}(1) = \frac{1}{1+e^{-1}} \approx 0.7311$$

$$\mathrm{SiLU}(-3) = \frac{-3}{1+e^{3}} \approx -0.1423$$

所以 SiLU 不会像 ReLU 那样把所有负值直接变成 0。

具体模型选择哪一种，取决于架构与训练设计。下一节的 SwiGLU 则进一步引入了两个分支之间的乘法。

10.7 现代 LLM 中常见的 SwiGLU：两条支路，逐元素相乘

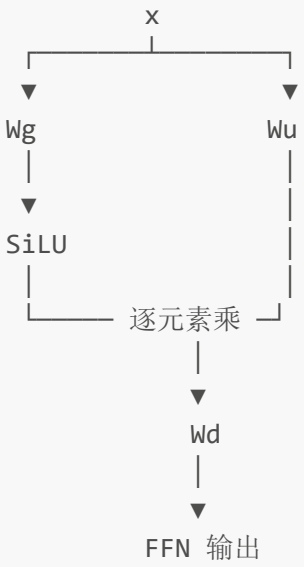
普通 FFN 可以理解为一条中间特征路径。SwiGLU 使用两条投影路径，其中一条经过 SiLU，再与另一条逐元素相乘：

$$g = \mathrm{SiLU}(xW_g)$$

$$u = xW_u$$

$$\boxed{\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = \big(g \odot u\big)W_d}$$

⊙ 表示逐元素相乘，不是矩阵乘法。



可以把 g 理解为根据输入产生的一组调节系数，u 是另一组中间特征。输入变化时，这些调节系数也会变化。

但这种“门”不是 0/1 开关，也不是一组概率：**SiLU 的输出可以为负，也可以大于 1。** 所以它可以缩小、放大，甚至反转另一支路某些分量的符号。

这种把门控结构用于 FFN 的设计，可对照 GLU Variants Improve Transformer 中的 SwiGLU 公式。

10.8 SwiGLU 也手算一次：看清“门”究竟做了什么

沿用输入：

$$x = [2, -1]$$

令 W_g 使用前面的 W_1，W_d 使用前面的 W_2，再增加一组权重：

$$W_u = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

门控支路先得到：

$$xW_g = [1, -3, 5]$$

经过 SiLU：

$$g \approx [0.7311, -0.1423, 4.9665]$$

另一条支路得到：

$$u = xW_u = [2, -1, 1]$$

逐元素相乘：

$$g \odot u \approx [1.4621,\ 0.1423,\ 4.9665]$$

注意第二维：负的调节系数乘上负的特征，得到正值。这正说明"门控"不等于"把值裁剪到 0 ~ 1"。

最后降维：

$$y = (g \odot u)W_d \approx [6.4287, -4.8243]$$

这个例子与普通 ReLU FFN 使用相同的输入，但多了一条输入相关的调节支路，因此表达形式不同。

10.9 参数量与计算量：为什么 FFN 往往占模型很大一部分？

忽略偏置，普通两矩阵 FFN 的参数量为：

$$P_{\text{FFN}} = DM + MD = 2DM$$

如果 M=4D：

$$P_{\text{FFN}} = 8D^2$$

SwiGLU 有三组矩阵，若中间维度为 M_g：

$$P_{\text{SwiGLU}} = DM_g + DM_g + M_gD = 3DM_g$$

因此不能简单地认为"SwiGLU 的中间维度也必须取 4D"。为了让参数量接近前面的 $8D^2$，可以选：

$$3DM_g \approx 8D^2 \quad \Longrightarrow \quad M_g \approx \frac{8}{3}D$$

用小整数核算：

结构	输入维度	中间维度	不计偏置的参数量
普通 FFN	12	48	$2 \times 12 \times 48 = 1152$

结构	输入维度	中间维度	不计偏置的参数量
SwiGLU	12	32	$3 \times 12 \times 32 = 1152$

真实模型会根据硬件对齐、容量和训练效果选择宽度，上面的相等只用于说明参数预算。

固定维度下，FFN 对一批序列的主要计算量随 B×T 线性增长，约为：

$$O(BTDM)$$

完整全注意力中的 Token 两两交互部分约为：

$$O(BT^2D)$$

后一式不含 Q/K/V 等投影的计算。不能只看这两个式子，就断定任何长度下 Attention 都比 FFN 更慢；实际耗时还取决于维度、内存访问和算子实现。

10.10 FFN 与 MoE、LoRA、RMSNorm、残差连接的关系

机制	和 FFN 怎样配合
RMSNorm	在常见 Pre-Norm 层中，先归一化隐藏状态，再交给 FFN
残差连接	把 FFN 的输出加回分支输入，形成更新后的隐藏状态
MoE	把一个共享的 FFN 换成多个 Expert，由 Router 为 Token 选择少数 Expert
LoRA	可以给 FFN 的线性投影加入低秩权重更新
KV Cache	通常保存 Attention 的历史 K/V，而不是 FFN 的中间激活

例如，在采用 Pre-Norm 的层中：

$$Y = H + \mathrm{FFN}(\mathrm{RMSNorm}(H))$$

这也解释了一个常见误区：FFN 的中间宽度 M 可以很大，但最后输出仍需回到 D 维，才能与 H 相加。

MoE 中的 Expert 可以采用普通 FFN，也可以采用 SwiGLU 等结构；“Expert”并不自动等于“一个人工指定领域的知识库”。

10.11 代码与练习：验证两种 FFN

下面的 NumPy 代码可以独立运行，分别复现普通 FFN 与 SwiGLU 的例子：

```
import numpy as np

W1 = np.array([[1.0, -1.0, 2.0],
               [1.0,  1.0, -1.0]])
W2 = np.array([[1.0,  0.0],
               [0.0,  1.0],
               [1.0, -1.0]])
Wu = np.array([[1.0, 0.0, 1.0],
```

```
                [0.0, 1.0, 1.0]])


def relu_ffn(x):
    expanded = x @ W1
    activated = np.maximum(expanded, 0)
    return activated @ W2


def silu(x):
    return x / (1 + np.exp(-x))


def swiglu_ffn(x):
    gate = silu(x @ W1)
    values = x @ Wu
    return (gate * values) @ W2


tokens = np.array([[2.0, -1.0], [1.0, 1.0]])
np.set_printoptions(precision=4, suppress=True)
print("普通 FFN 的两个 Token 输出：")
print(relu_ffn(tokens))
print("第一个 Token 的 SwiGLU 输出：")
print(swiglu_ffn(tokens[0]))
print("不使用激活函数时，第一个 Token 的输出：")
print((tokens[0] @ W1) @ W2)
```

输出约为：

```
普通 FFN 的两个 Token 输出：
[[ 6. -5.]
 [ 3. -1.]]
第一个 Token 的 SwiGLU 输出：
[ 6.4287 -4.8243]
不使用激活函数时，第一个 Token 的输出：
[ 6. -8.]
```

这里展示前向计算；“前馈”描述计算连接的方向，并不表示 FFN 不能用反向传播训练。NumPy 示例没有实现训练过程。

练习 1： 输入形状为 `[2,5,4]`，中间宽度为 12，两个权重矩阵和输出分别是什么形状？

答案：W_1 为 `[4,12]`，W_2 为 `[12,4]`，中间表示为 `[2,5,12]`，输出为 `[2,5,4]`。

练习 2： 忽略偏置，这个普通 FFN 有多少参数？

$$4 \times 12 + 12 \times 4 = 96$$

练习 3： 如果只改变第二个 Token 进入 FFN 时的向量，第一个 Token 的本次 FFN 输出会变吗？

答案：对这里固定权重、无随机操作的逐位置 FFN，不会。若改变的是原始输入句子，并重新运行之前的 Attention，则第一个 Token 的隐藏状态也可能随之改变。

> **FFN 对每个 Token 已有的特征做共享参数的非线性变换；Attention 提供跨 Token 的信息交互，两者共同形成 Transformer 的表达能力。**

---

11. 残差连接：保留原输入，再叠加本层学到的更新

> 学习目标：能手算残差相加，理解“学习更新量”与梯度直通路径，分清 Pre-Norm、Post-Norm，以及残差连接和归一化各自的作用。
>
> 第 3 章和前面的总流程已经多次出现“加回 X”“加回 H”。这一章把这些加号展开解释。

11.1 什么叫残差连接？先看最短的公式

假设一个模块原本计算：

$$y = F(x)$$

加入残差连接后，变为：

$$\boxed{y = x + F(x)}$$

输入走两条路：



一条路经过 F，另一条路直接到达相加位置。直接连接输入与后续输出的路径，也叫跳跃连接或 shortcut。

可以把它理解为修改一份已经有内容的草稿：保留现有版本，再加上这一轮的修改，而不是每次都从空白重写。但在实际神经网络中，这份“草稿”是隐藏向量，“修改”也是一个向量。

11.2 为什么叫“残差”？学的是相对于输入的变化量

假设我们希望模块最终实现某个目标变换 G：

$$y = G(x)$$

残差形式把它写成：

$$G(x) = x + F(x)$$

于是：

$$F(x) = G(x) - x$$

这里的 F 表示目标输出相对于原输入的变化量，因此称为残差分支。

例如：

$$x = [2, 1]$$

希望得到：

$$G(x) = [2.1, 0.9]$$

那残差分支只需输出：

$$F(x) = [0.1, -0.1]$$

再相加：

$$[2, 1] + [0.1, -0.1] = [2.1, 0.9]$$

如果当前阶段保留原特征就已经合适，残差分支可以接近零：

$$F(x) \approx 0 \quad \Longrightarrow \quad y \approx x$$

这种把恒等映射作为容易表达的情况、再学习偏离量的设计，可联系 Deep Residual Learning for Image Recognition 中的残差学习思想。

但"残差"不是损失函数，也不是训练程序事先给每层算好的标准答案。模型仍然通过最终训练目标和反向传播学习 F，真实网络中的更新也不一定很小。

11.3 完整向量例子：相加，不是替换或拼接

沿用 RMSNorm 章节的 4 维向量：

$$x = [1, 2, 2, 1]$$

假设某个分支输出：

$$F(x) = [0.1, -0.2, 0.3, 0]$$

残差相加得到：

$$y = [1 + 0.1,\ 2 - 0.2,\ 2 + 0.3,\ 1 + 0]$$

$$\boxed{y = [1.1, 1.8, 2.3, 1]}$$

最后一维没有变化，因为分支在这一维给出的更新为 0。

这一步是逐元素相加，输入与输出维度都为 4。它不等于把两段向量拼接：

$$\mathrm{Concat}(x, F(x)) = [1, 2, 2, 1, 0.1, -0.2, 0.3, 0]$$

拼接后变成 8 维，属于另一种操作。

操作	对 4 维 x 和 4 维 F(x) 的结果	含义
替换：$F(x)$	4 维	只保留分支输出
残差相加：$x + F(x)$	4 维	原表示与分支更新逐维叠加
拼接	8 维	两份表示并排存放

因此，普通残差相加要求两边形状匹配。若想连接不同维度的模块，需要先做明确的投影对齐，例如：

$$y = xW_s + F(x)$$

此时 shortcut 上的 W_s 是可学习投影，已经不再是完全不做变换的恒等路径。常见 Transformer 子层保持隐藏维度不变，就是为了方便直接相加。

11.4 为什么深层网络需要这条路？从梯度看

训练时，后面的误差需要通过链式法则传回前面的层。先只看标量，避免矩阵符号挡住重点。

没有残差时：

$$y = F(x) \quad \Longrightarrow \quad \frac{\partial y}{\partial x} = F'(x)$$

有残差时：

$$y = x + F(x) \quad \Longrightarrow \quad \boxed{\frac{\partial y}{\partial x} = 1 + F'(x)}$$

这里多出来的 1，来自直接传递 x 的那条路径。

如果损失是 $\mathcal{L}$：

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y}\big(1 + F'(x)\big)$$

即使某个位置上 F 的导数很小，直接路径仍能为梯度提供一项贡献。

看一个刻意简化的例子：每层都取 $F(x) = 0.01x$，连续堆 20 层。

普通串联：

$$x_{\ell+1} = 0.01x_\ell$$

$$\frac{\partial x_{20}}{\partial x_0} = 0.01^{20} = 10^{-40}$$

残差形式：

$$x_{\ell+1} = x_\ell + 0.01x_\ell = 1.01x_\ell$$

$$\frac{\partial x_{20}}{\partial x_0} = 1.01^{20} \approx 1.2202$$

这个例子展示的是：在分支导数较小的情况下，恒等路径能够显著改变梯度连乘的行为。真实网络包含非线性、归一化和复杂矩阵，不会始终表现得像这个标量例子。

如果熟悉向量求导，对应表达就是：

$$J_y = I + J_F$$

$$\nabla_x \mathcal{L} = (I + J_F)^T \nabla_y \mathcal{L}$$

I 是恒等矩阵，对应那条直接连接。这里的 J_F 表示 F 对输入的雅可比矩阵。

11.5 有残差，就一定不会梯度消失或爆炸吗？

不一定。“有一条直接路径”不等于“总梯度永远等于 1”。

两个反例就能说明：

反例 1：分支刚好抵消输入。

$$F(x) = -x \quad \Longrightarrow \quad y = x - x = 0$$

$$\frac{\partial y}{\partial x} = 1 - 1 = 0$$

此时总梯度仍然为零。

反例 2：每一层都把输入再加一遍。

$$F(x) = x \quad \Longrightarrow \quad y = 2x$$

连续 L 层后，导数为：

$$2^L$$

仍然可能迅速增大。

因此，残差连接需要与初始化、归一化、学习率等设计共同工作。它提供了更容易保留信息、传播梯度的结构条件，并不是独立的稳定性保证。

从前向传播也能看到同样的问题：

$$x_{\ell+1} = x_\ell + F_\ell(x_\ell)$$

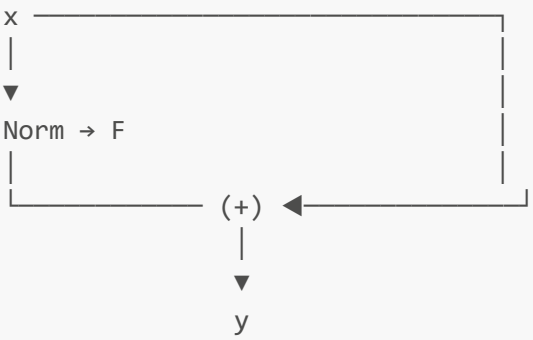
如果多个分支持续输出同方向的大更新，隐藏状态的尺度就可能增长。这也呼应了前面为什么要学习 RMSNorm。

11.6 和 RMSNorm 放在一起：Pre-Norm、Post-Norm 到底差在哪？

Norm 表示选定的归一化方法，可以是 LayerNorm 或 RMSNorm。Pre、Post 说的是它的位置。

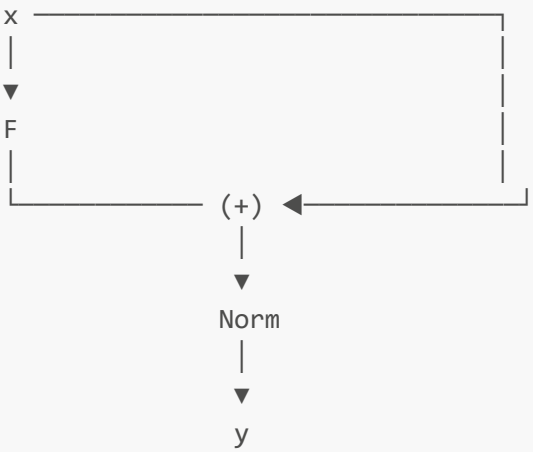
Pre-Norm：先归一化分支输入，分支输出再加回原输入。

$$\boxed{y = x + F(\mathrm{Norm}(x))}$$



Post-Norm：先计算分支并做残差相加，再对相加结果归一化。

$$\boxed{y = \mathrm{Norm}(x + F(x))}$$



两种式子通常不等价，因为归一化会依据它接收到的向量重新计算统计量。

在 Pre-Norm 中，原输入 x 有一条直接到加号输出的恒等路径；在 Post-Norm 中，即使沿 shortcut 走到加号，之后仍要经过 Norm。

这解释了两者梯度路径上的结构差异，但不能据此断言任何任务上 Pre-Norm 都必然更好。有关归一化位置与训练行为的分析，可参见 On Layer Normalization in the Transformer Architecture。

本讲义的主要流程采用 Pre-Norm。特别注意，不要误写成：

$$\mathrm{Norm}(x) + F(\mathrm{Norm}(x))$$

这个写法改变了 shortcut 保留的内容，已经不是本章所描述的 $x + F(\mathrm{Norm}(x))$。

11.7 一层 Transformer 为什么通常有两个残差相加?

因为一层中通常有 Attention 和 FFN 两个主要子层，每个子层各自产生一次更新。

对于本讲义采用的 Pre-Norm 结构：

$$u_1 = \mathrm{RMSNorm}_1(X)$$

$$A = \mathrm{Attention}(u_1)$$

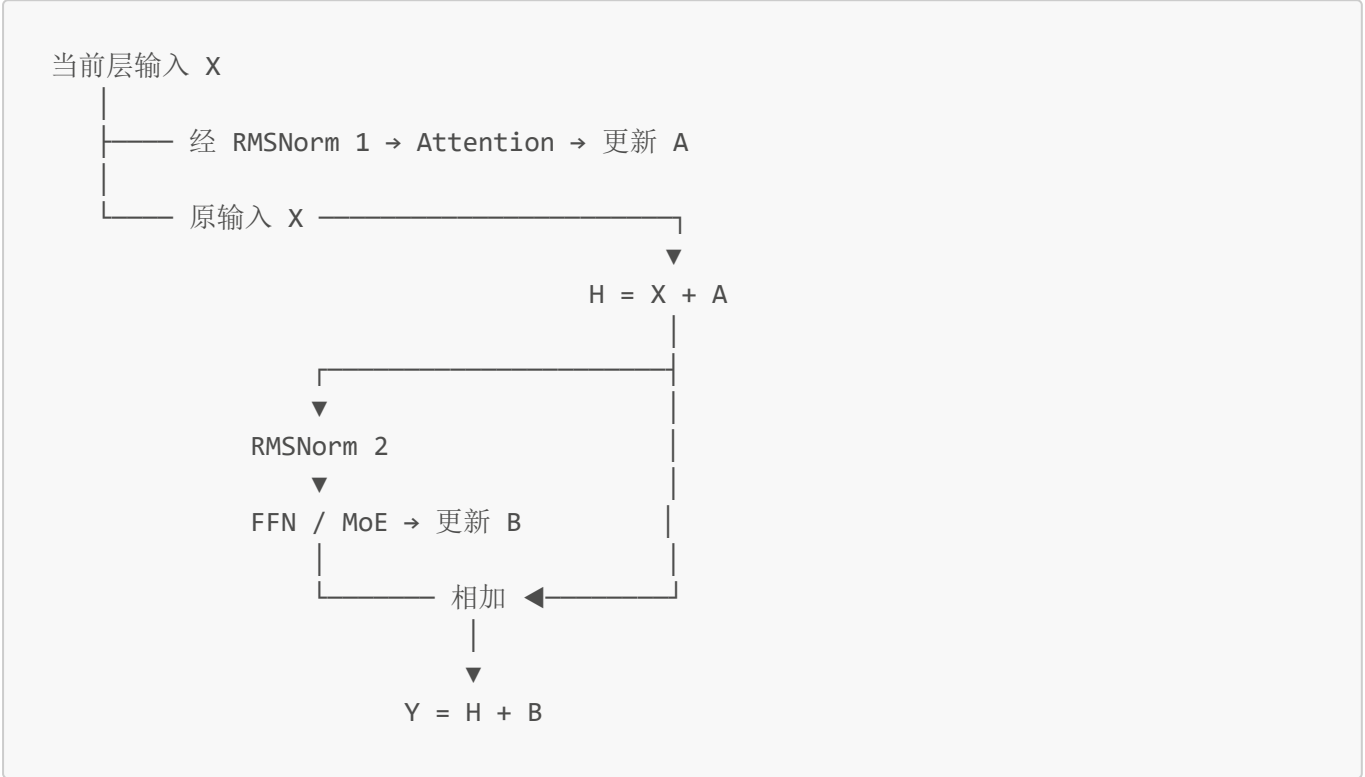
$$H = X + A$$

$$u_2 = \mathrm{RMSNorm}_2(H)$$

$$B = \mathrm{FFN}(u_2)$$

$$Y = H + B$$

其中 A 包含 Attention 的输出投影，不是尚未合并、投影的某个单 Head 输出。



第二次要加回的是 H，不是再次加回最初的 X。因为 H 已经包含 Attention 对当前层输入的更新。

从代数上看，这一层输出可以写成：

$$Y = X + A + B$$

但 A、B 不是与输入无关的固定常量；尤其 B 是根据 H 重新计算的，所以计算顺序不能随意交换。

11.8 把 FFN 和残差连起来，完整算一次

使用第 10 章的二维 FFN 例子。当前 Token 的原始隐藏状态为：

$$X = [2, -1]$$

为了集中看清残差与 FFN，**假设 Attention 已经根据归一化后的输入算出分支更新**：

$$A = [1, -1]$$

这里 A 是已知的教学输入，不是在省略 Q/K/V 权重的情况下推算出来的真实 Attention 输出。

第一步：Attention 残差相加。

$$H = X + A = [3, -2]$$

第二步：对 H 做 RMSNorm。

令 gamma 全为 1，手算忽略很小的稳定项：

$$\mathrm{RMS}(H) = \sqrt{\frac{3^2 + (-2)^2}{2}} = \sqrt{6.5} \approx 2.5495$$

$$u_2 = \mathrm{RMSNorm}(H) \approx [1.1767, -0.7845]$$

第三步：使用第 10 章相同的 W_1、ReLU、W_2。

$$u_2 W_1 \approx [0.3922, -1.9612, 3.1379]$$

$$\mathrm{ReLU}(u_2 W_1) \approx [0.3922, 0, 3.1379]$$

$$B = \mathrm{FFN}(u_2) \approx [3.5301, -3.1379]$$

第四步：FFN 残差相加。

$$Y = H + B$$

$$\boxed{Y \approx [6.5301, -5.1379]}$$

这个结果故意没有做成“很小的更新”，以提醒我们：残差分支能输出多大的数，由实际权重和输入决定。残差连接本身不会强制更新很小，也不会自动把 Y 归一化成单位尺度。

11.9 实际代码中的加号，应该怎样读？

下面用 NumPy 验证上一节的完整计算。Attention 更新作为已知量传入；代码实现的是 Attention 之后的残差、RMSNorm、FFN 和第二次残差，不包含真实的注意力计算：

```
import numpy as np


def rms_norm(x, eps=1e-6):
    # 本例 gamma 全为 1，沿最后一个特征轴归一化
    return x / np.sqrt(np.mean(x ** 2, axis=-1, keepdims=True) + eps)


def ffn(x):
    W1 = np.array([[1.0, -1.0, 2.0],
                   [1.0,  1.0, -1.0]])
    W2 = np.array([[1.0,  0.0],
                   [0.0,  1.0],
                   [1.0, -1.0]])
    return np.maximum(x @ W1, 0) @ W2


x = np.array([2.0, -1.0])
attention_update = np.array([1.0, -1.0])  # 假设已算好的 Attention 输出
```

```
h = x + attention_update                    # 第一次残差，保留 x
normalized_h = rms_norm(h)
ffn_update = ffn(normalized_h)
y = h + ffn_update                          # 第二次残差，保留 h

np.set_printoptions(precision=4, suppress=True)
print("第一次相加 H：", h)
print("归一化后：", normalized_h)
print("FFN 更新：", ffn_update)
print("最终输出 Y：", y)
print("普通 20 层的导数：", 0.01 ** 20)
print("残差 20 层的导数：", round(1.01 ** 20, 4))
```

输出约为：

```
第一次相加 H： [ 3. -2.]
归一化后： [ 1.1767 -0.7845]
FFN 更新： [ 3.5301 -3.1379]
最终输出 Y： [ 6.5301 -5.1379]
普通 20 层的导数： 1e-40
残差 20 层的导数： 1.2202
```

下面是对应实际模型调用顺序的伪代码；其中各模块已经在别处定义：

```
x = 输入隐藏状态

residual = x
x = attention(rmsnorm_1(x))
x = residual + x

residual = x
x = ffn(rmsnorm_2(x))
x = residual + x
```

注意两次 `residual = x` 保存的是不同阶段的隐藏状态。

11.10 常见问题与练习

问题 1：残差连接本身需要新增多少可学习参数?

如果只是同形状的逐元素相加、shortcut 为恒等映射，不需要新增可学习参数。F 内部当然仍然有参数；若 shortcut 上增加投影或缩放参数，则另当别论。

问题 2：它会把隐藏状态维度翻倍吗?

不会。相加后仍是相同维度；拼接才会把两个向量并排放在一起。

问题 3：残差连接和 KV Cache 都是在“保留信息”，是同一回事吗?

不是。残差连接把当前计算阶段的隐藏状态传给后续加法；KV Cache 在自回归推理中保存历史 Token 的 K/V，供后续生成步骤读取。它们保留的对象、跨越的计算范围和用途都不同。

问题 4：FFN 的输出等于整层输出吗?

通常不等于。在本讲义的 Pre-Norm 结构中，还要把 FFN 输出加回 H，才得到该层的输出。

练习 1： $x = [1, 2]$，$F(x) = [-1, 0.5]$，残差输出是什么?

$$x + F(x) = [0, 2.5]$$

练习 2： 在 $y = x + F(x)$ 中，如果 F 是恒等于零的函数，输出和对输入的导数分别是什么?

答案：输出仍为 x；标量导数为 1，向量雅可比为 I。

练习 3： 输入为 `[B,T,4]`，分支输出为 `[B,T,8]`，能否按本章的普通残差连接直接相加?

答案：不能。需要让分支降回 4 维，或用明确的投影把 shortcut 对齐到 8 维；常见 Transformer FFN 采用前一种方式。

> **残差连接让每个分支在已有隐藏状态上叠加更新，并提供直接的信息与梯度路径；归一化负责调节尺度，Attention 和 FFN 负责计算更新内容。**

---

12. 可选：MoE（Mixture of Experts）

12.1 从普通 FFN 过渡到 MoE

第 10 章已经详细介绍 FFN。沿用行向量写法，忽略偏置时：

$$\mathrm{FFN}(x) = \phi(xW_1)W_2$$

同一层内，所有 Token 共享这一个 FFN 的参数，分别计算各自的输出。FFN 也可以采用前面讲过的 SwiGLU 结构。

MoE 接下来改变的是“让当前 Token 经过哪几个 FFN / Expert”，而不是取消特征的非线性变换。

12.2 MoE 的思想

Mixture of Experts：

> 不再只有一个 FFN，而是有多个 Expert。

例如：

```
Expert 1
Expert 2
```

```
Expert 3
Expert 4
Expert 5
Expert 6
Expert 7
Expert 8
```

但是每个 Token 不会经过全部 Expert。

Router 会选择其中少量 Expert。

例如 Top-2：

```
Token "Python"
      │
      ▼
    Router
      │
      ├── Expert 2: 0.72
      └── Expert 7: 0.21
```

最后：

$$y = 0.72E_2(x) + 0.21E_7(x)$$

12.3 一个直观例子

假设有四个专家：

```
Expert 1：数学
Expert 2：代码
Expert 3：语言
Expert 4：历史
```

输入：

```
def quick_sort(...)
```

Router 可能给：

```
代码 Expert = 0.82
数学 Expert = 0.11
```

```
语言 Expert = 0.04
历史 Expert = 0.03
```

Top-2 Router 只选择：

```
Expert 2
Expert 1
```

因此虽然模型拥有四个 Expert 的参数：

$$P_{\text{total}}$$

一次前向传播实际只激活其中一部分：

$$P_{\text{active}} \ll P_{\text{total}}$$

12.4 MoE 的核心优势

假设：

```
8 个 Expert
每个 Expert = 1B 参数
每个 Token 只激活 2 个 Expert
```

模型总 Expert 参数：

$$8B$$

但每个 Token 实际参与计算：

$$2B$$

因此可以：

> 增大模型参数容量，但不让每个 Token 的计算量同比增长。

12.5 MoE 的问题

MoE 不是免费午餐。

主要问题包括：

1. Router Load Imbalance
2. Expert Hotspot

3. All-to-All Communication
4. Expert Parallelism
5. Capacity Overflow

例如：

```
1000 个 Token
```

如果 Router 都选择：

```
Expert 2
```

则：

```
Expert 2: 900 Token
Expert 1: 30 Token
Expert 3: 40 Token
Expert 4: 30 Token
```

就会出现严重负载不均衡。

因此 MoE 通常需要：

$$Auxiliary\ Loss$$

鼓励 Token 更均匀地分配到不同 Expert。

12.6 一句话总结

> **MoE 用“稀疏激活”换取更大的模型容量：模型拥有很多 Expert，但每个 Token 只使用少数 Expert。**

---

13. 可选：LoRA

> 本章先理解低秩更新的形状与参数量；完整训练流程、初始化、梯度手算和代码见第 22 章。SFT 与 LoRA 的关系先看第 21.1 节。

13.1 为什么需要 LoRA?

假设一个 7B 模型：

$$7 \times 10^9$$

个参数。

如果完整 Fine-tuning，需要：

- 参数
- 梯度
- Optimizer State

显存开销很大。

LoRA 的思想是：

> 冻结原始权重，只训练一个很小的低秩增量。

13.2 普通 Linear

原始：

$$y = Wx$$

其中：

$$W \in \mathbb{R}^{d_{out} \times d_{in}}$$

Full Fine-tuning：

$$W \to W + \Delta W$$

需要训练完整：

$$\Delta W$$

13.3 LoRA 的核心

LoRA 假设：

$$\Delta W$$

可以近似为两个低秩矩阵：

$$\Delta W = BA$$

其中：

$$A \in \mathbb{R}^{r \times d_{in}}$$

$$B \in \mathbb{R}^{d_{out} \times r}$$

并且：

$$r \ll d_{in}, d_{out}$$

最终：

$$y = Wx + BAx$$

训练时：

```
W：冻结
A：训练
B：训练
```

13.4 参数量例子

假设：

$$W \in \mathbb{R}^{4096 \times 4096}$$

原始参数量：

$$4096^2 = 16,777,216$$

约：

$$16.8M$$

如果：

$$r = 8$$

LoRA 参数：

$$A : 8 \times 4096$$

$$B : 4096 \times 8$$

总参数：

$$8 \times 4096 + 4096 \times 8$$

$$= 65,536$$

只相当于原矩阵：

$$\frac{65536}{16777216} \approx 0.39\%$$

13.5 LoRA 为什么叫 Low-Rank?

因为：

$$\text{rank}(BA) \le r$$

如果：

$$r = 8$$

那么：

$$\Delta W$$

最多只有 rank 8。

LoRA 的核心假设：

> 大模型在针对下游任务进行微调时，需要改变的有效参数子空间可能远低于完整参数维度。

13.6 LoRA 一般加在哪里？

Transformer 中常见：

```
Attention:
Wq
Wk
Wv
Wo

MLP:
W_up
W_gate
W_down
```

例如只对：

```
Wq
Wv
```

加入 LoRA：

$$W_q' = W_q + B_q A_q$$

$$W_v' = W_v + B_v A_v$$

13.7 LoRA 的缩放系数

实际 LoRA 常写成：

$$y = Wx + \frac{\alpha}{r} BAx$$

其中：

$$\alpha$$

为 LoRA Alpha。

例如：

```
r = 8
alpha = 16
```

则：

$$\frac{\alpha}{r} = 2$$

即：

$$y = Wx + 2BAx$$

13.8 一句话总结

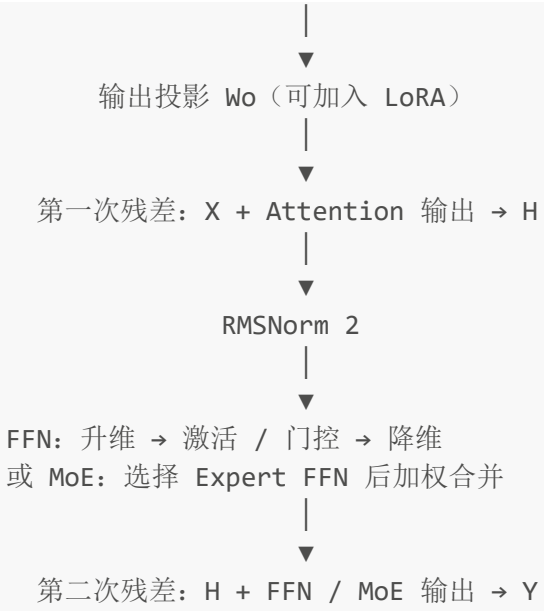
> **LoRA 不修改大模型原始权重，而是学习一个低秩的权重增量，从而显著减少可训练参数和微调显存。**

---

14. 十个概念放到一张图里理解

下面采用 Pre-Norm 结构。Q/K Norm、YaRN、GQA 是否启用由模型决定；“RoPE / YaRN”表示选择相应的位置编码配置，不是做两次位置旋转。

```
当前层输入 X
    │
    ▼
RMSNorm 1
    │
    ▼
Wq / Wk / Wv（可加入 LoRA）
    │
    ├── Q → 可选 Q Norm → RoPE / YaRN ──────────────┐
    │                                               │
    ├── K → 可选 K Norm → RoPE / YaRN → 缓存 K ─────┤
    │                                               │
    └── V ──────────────────────────→ 缓存 V ────────┤
                                                    ▼
                                  Attention（可采用 GQA 共享 K/V）
```



如果采用 YaRN，幅度缩放按模型实现合并在位置编码或注意力计算的等价位置，避免重复缩放。LoRA 还可以作用于 FFN 或 MoE Expert 的线性层。

下面省略 LoRA 缩放系数，只表示低秩更新的位置：

$$W_q \to W_q + B_q A_q$$

---

15. 一个完整的小例子

假设现在模型处理：

```
I love large language models
```

并生成下一个 Token。这里为了说明流程，把每个英文单词视为一个 Token；假设前四个 Token 的 K/V 已经缓存，当前处理 models。实际分词结果由 Tokenizer 决定。

Step 0：先用 RMSNorm 调整隐藏状态的尺度

在采用 Pre-Norm 的模型中，每个 Token 的 Hidden State 先独立进行 RMSNorm，再送入 Q/K/V 投影。

例如，`models` 在当前层的隐藏状态若简化为：

$$x_{\text{models}} = [1, 2, 2, 1]$$

忽略稳定项且 gamma 全为 1 时：

$$\widetilde{x}_{\text{models}} \approx [0.6325, 1.2649, 1.2649, 0.6325]$$

下面的 Q/K/V 由这些归一化后的特征计算得到。

Step 1：计算 Q/K/V

最后一个 Token：

```
models
```

得到：

$$Q_{\text{models}}$$

当前序列对应的 K/V 为（历史部分直接复用缓存）：

$$K_0, K_1, K_2, K_3, K_4$$

$$V_0, V_1, V_2, V_3, V_4$$

Step 2：Q/K Normalization

$$Q_i \to \hat{Q}_i$$

$$K_i \to \hat{K}_i$$

避免 Q/K 模长越来越大导致 Attention Logit 爆炸。

Step 3：RoPE，或采用 YaRN 调整后的配置

Token Position：

```
I         0
love      1
large     2
language  3
models    4
```

因此：

$$Q_4 \to R(4\theta)Q_4$$

$$K_i \to R(i\theta)K_i$$

使 Attention 知道 Token 间的相对距离。

如果此模型采用 YaRN，则把这里的频率换成调整后的频率，并按模型约定应用幅度因子。以所有 Q/K 维度都参与旋转和统一缩放的示意形式表示：

$$Q_4 \to aR(4\theta')Q_4, \qquad K_i \to aR(i\theta')K_i$$

这里代替的是上面的普通 RoPE 变换，不能把两种变换再连续做一遍。频率与温度的数值例子见第 6 章。

Step 4：GQA

假设：

```
Q Heads  = 32
KV Heads = 8
```

每 4 个 Query Head 共用：

```
1 个 K Head
1 个 V Head
```

减少 K/V 数量。

Step 5：KV Cache

历史：

```
K0,K1,K2,K3
V0,V1,V2,V3
```

已经存在 Cache。

只需要计算新 Token：

```
K4,V4
```

并追加到 Cache。

Step 6：Attention

新的 Query 与全部缓存 Key：

$$Q_4[K_0,K_1,K_2,K_3,K_4]^T$$

得到 Attention Score。

再：

$$O_4=\sum_i \alpha_i V_i$$

Step 7：Attention 的残差相加

Attention 经过输出投影后，加回进入当前层时的原始隐藏状态：

$$H = X + O_{\text{attn}}$$

这里的加号表示逐元素相加，并不改变隐藏维度。它保留了 X，并叠加 Attention 从上下文中计算出的更新。

Step 8：RMSNorm、FFN / MoE，再做第二次残差相加

先对 H 做归一化，再把每个 Token 的特征送入 FFN：

$$u = \mathrm{RMSNorm}(H)$$

普通 FFN 可使用：

$$B = \phi(uW_1 + b_1)W_2 + b_2$$

SwiGLU 则可使用：

$$B = \big(\mathrm{SiLU}(uW_g) \odot (uW_u)\big)W_d$$

如果采用 MoE，则由 Router 为 Token 选择 Expert，并合并相应输出。最后加回的是 H：

$$Y = H + B$$

FFN 的两种矩阵手算见第 10 章；包含两次残差与 RMSNorm 的数值例子见第 11.8 节。

Step 9：如果使用 LoRA 微调

例如：

$$Q = \widetilde{X}W_Q + \frac{\alpha}{r}(\widetilde{X}A_Q^\top)B_Q^\top$$

这里沿用前文按行存放 Token 的写法；W_Q 的形状是 $d_{in} \times d_{out}$，A_Q, B_Q 的形状沿用第 13 章，因此 LoRA 分支需要相应的转置。第 22.6 节的 PyTorch 代码采用同样的行向量批处理方式。

原始：

$$W_Q$$

保持冻结，只训练：

$$A_Q, B_Q$$

---

16. 最容易混淆的几个问题

16.1 Q/K Normalization 和 RMSNorm 是一回事吗？

这两个名字描述的角度不同：**RMSNorm 是“怎么归一化”，Q/K Normalization 是“对谁归一化”。**

RMSNorm 常用于 Attention / FFN 前的 Hidden States，也可以作为 Q/K Norm 采用的具体算法：

```
Hidden States
    ↓
RMSNorm（调节隐藏状态的尺度）
    ↓
Wq / Wk / Wv
    ↓
Q / K / V
    ↓
可选的 Q/K Norm（可以选择 RMSNorm）
    ↓
RoPE、Attention
```

即使隐藏状态已经做过 RMSNorm，投影矩阵仍然可以放大 Q/K，所以这两个位置上的归一化不能简单视为重复。详细例子见第 3.9 节。

16.2 RoPE 是加到 Embedding 上的吗?

通常不是。

经典位置编码可能是：

$$X' = X + PE$$

而 RoPE 主要作用于：

$$Q, K$$

即：

$$Q' = \mathrm{RoPE}(Q)$$

$$K' = \mathrm{RoPE}(K)$$

16.3 GQA 会减少 Q Head 吗?

不会。

例如：

```
Q Head  = 32
KV Head = 8
```

仍然有 32 个 Query Head。

减少的是：

$$K/V\ Head$$

16.4 KV Cache 为什么不能缓存 Attention Output?

因为下一步新 Query 需要重新和所有历史 Key 计算 Attention：

$$Q_{t+1}K_{1:t}^T$$

Attention 权重取决于新的：

$$Q_{t+1}$$

所以旧的 Attention Output 不能直接替代 K/V。

16.5 KV Cache 为什么随着序列增长越来越大?

每生成一个 Token，都需要加入：

$$K_t, V_t$$

所以：

$$M_{KV} \propto T$$

Sequence Length 翻倍：

$$KV\ Cache$$

通常也近似翻倍。

16.6 MoE 会减少模型总参数吗?

不会。

MoE 的总参数可能非常大。

它减少的是：

> 每个 Token 实际激活、参与计算的参数。

所以应区分：

$$Total\ Parameters$$

和：

$$Active\ Parameters$$

16.7 LoRA 会减少推理模型参数吗?

训练时 LoRA 大幅减少：

$$Trainable\ Parameters$$

推理时可以：

1. 保留 LoRA Adapter 独立计算；
2. 将：

$$BA$$

Merge 到：

$$W$$

变成：

$$W' = W + \frac{\alpha}{r} BA$$

然后正常推理。

16.8 YaRN 是把文本压缩成更少的 Token 吗？

不是。它调整位置编码频率和注意力尺度，Token 数量仍然保留。序列变长通常还会增加 KV Cache 与 Attention 计算成本，见第 6.7 节。

16.9 FFN 会直接把不同 Token 的特征混在一起吗？

常见的逐位置 FFN 不会：每个 Token 独立使用同一层的权重。但它接收的隐藏状态通常已经包含 Attention 汇集的上下文，见第 10.5 节。

16.10 残差连接能完全避免梯度消失或爆炸吗？

不能。它提供了直接路径，但总梯度仍由直接路径与分支共同决定。分支抵消输入或持续放大输入时，仍然可能出问题；反例见第 11.5 节。

---

17. 面试 / PPT 最值得记住的十一句话

Q/K Normalization

> 对 Q 和 K 做归一化，主要是为了控制 Attention Logit 的尺度，防止 Softmax 过度饱和，提高训练稳定性。

RMSNorm

> RMSNorm 不减均值，而是按均方根缩放特征，再逐维乘可学习参数；常在 Attention / FFN 前对每个 Token 的隐藏向量独立计算。

残差连接

> 残差连接把分支输出加回原隐藏状态，使网络可以在已有表示上学习更新，并提供直接的梯度路径。

FFN

> FFN 用共享参数逐 Token 地加工特征，通过线性投影、非线性激活或门控，再映射回隐藏维度。

GQA

> GQA 让多个 Query Head 共享较少的 K/V Head，是 MHA 和 MQA 的折中，可以显著减少 KV Cache。

RoPE

> RoPE 根据 Token Position 对 Q/K 进行旋转，使 QK 点积自然包含两个 Token 的相对位置信息。

YaRN

> YaRN 通过分频段调整 RoPE 并调节注意力尺度来支持上下文扩展；更长的有效上下文仍需模型适应与评估。

KV Cache

> KV Cache 在自回归推理中保存历史 Token 的 K/V，使 Decode 时不必重复计算历史 Token 的 K/V，用显存换计算。

MoE

> MoE 使用 Router 为每个 Token 选择少量 Expert，在扩大总模型容量的同时控制每个 Token 的实际计算量。

LoRA

> LoRA 将权重更新表示为低秩矩阵 BA，冻结原模型，只训练少量低秩参数，实现参数高效微调。

SFT

> SFT 用参考回答构造监督，通过因果预测和交叉熵学习回答行为；它既可以更新全模型，也可以只更新 LoRA 参数。

---

18. 结构、推理与训练之间的关系

可以从五个方面理解。

训练稳定性

残差连接提供保留隐藏状态与传播梯度的直接路径；RMSNorm 常在 Attention / FFN 入口调节特征尺度；Q/K Normalization 进一步调节投影后的 Q/K。这些机制可以共同使用。

主要对应：

$$\boxed{\text{Residual} + RMSNorm + Q/K\ Normalization}$$

Attention 与位置

RoPE 编码位置关系；YaRN 调整其频率与注意力尺度，帮助模型适应更长上下文。

主要对应：

$$\boxed{RoPE + YaRN}$$

推理效率与显存

主要对应：

$$\boxed{GQA + KV\ Cache}$$

其中：

$$GQA$$

减少每个 Token 需要保存的 K/V 数量，

而：

$$KV\ Cache$$

减少历史 Token 的重复计算。

特征变换与模型容量

FFN 对每个 Token 的特征进行非线性加工；MoE 用多个 Expert 扩大这部分的容量，并让每个 Token 只选择少数 Expert。

主要对应：

$$\boxed{FFN + MoE}$$

通过稀疏激活 Expert：

$$Total\ Params \uparrow$$

但：

$$Active\ Params$$

不同比例增长。

监督微调与训练成本

SFT 决定模型根据哪些示范、哪些目标 Token 学习；LoRA 通过低秩参数更新降低微调成本。两者配合时，使用同一套 SFT 损失，只把可训练参数主要限制为各层的低秩矩阵。

主要对应：

$$\boxed{\text{SFT 训练目标} + \text{LoRA 参数更新}}$$

通过：

$$\Delta W = BA$$

减少可训练参数。

---

19. 最终总图

```
Token → Embedding
          │
          ▼
       当前层输入 X
          │
          ▼
       RMSNorm 1：调节特征尺度
          │
          ▼
       Attention：跨 Token 汇集信息
       ├── Q/K Norm：可选的 Q/K 尺度控制
       ├── RoPE：位置编码
       │    └── YaRN：可选的长上下文扩展配置
       ├── GQA：共享 K/V Head
       └── KV Cache：保存历史 K/V
          │
          ▼
       第一次残差：加回 X → H
          │
          ▼
       RMSNorm 2：调节特征尺度
          │
          ▼
       FFN：逐 Token 非线性加工
       ├── 普通 FFN / SwiGLU
       └── MoE：按路由选择多个 Expert
          │
          ▼
       第二次残差：加回 H → 下一层

LoRA：可以用于 Attention / FFN / MoE 中的线性层。
上图采用 Pre-Norm；不同模型的归一化位置可能不同。
```

训练时，可以在模型最后一层后接上：



生成时使用的历史 KV Cache 与训练中的激活保存用途不同；第 22 章会解释为什么 LoRA 训练仍需要显存。

---

20. 建议学习顺序

如果第一次系统学习这些技术，建议：

```
1. Self-Attention
      ↓
2. Multi-Head Attention
      ↓
3. FFN / SwiGLU
      ↓
4. 残差连接
      ↓
5. LayerNorm / RMSNorm
      ↓
6. RoPE
      ↓
7. YaRN
      ↓
8. KV Cache
      ↓
9. GQA / MQA
      ↓
10. Q/K Normalization
      ↓
11. MoE
      ↓
12. SFT：数据、标签错位与监督损失
      ↓
13. LoRA：低秩原理与训练更新
```

学习训练部分时，先读第 21 章，再结合第 13 章的矩阵形状阅读第 22 章，最后完成第 22 章中的 LLaMA-Factory 实训衔接。

理解 FFN、残差连接与归一化如何组成一层之后，再重点关注下面三个与推理紧密相关的机制：

$$\boxed{RoPE + KV\ Cache + GQA}$$

这三个概念直接联系位置表示、历史缓存与 K/V 共享；学习长上下文扩展时，再结合 YaRN 理解位置尺度与缓存成本。

如果进一步研究 LLM Serving，则应继续学习：

```
KV Cache
   ↓
PagedAttention
   ↓
Continuous Batching
   ↓
KV Cache Eviction
   ↓
KV Cache Offloading
   ↓
Prefix Caching
   ↓
Disaggregated Prefill / Decode
```

这些技术基本都是围绕：

> **如何管理计算、显存、KV Cache 和请求调度**

展开的。

---

21. SFT：让模型根据示范学习怎样回答

> 学习目标：能把一条问答数据变成训练样本，手算回答部分的交叉熵，解释标签错位、掩码、反向传播与参数更新。
>
> 第 1～20 章主要回答“模型怎样计算”；本章开始回答“用什么数据、怎样让模型学会任务”。第 22 章再把同一套 SFT 流程换成 LoRA 参数更新。

21.1 预训练、SFT、LoRA 分别在说什么？

SFT 是 **Supervised Fine-Tuning，监督微调。**在这里，我们讨论自回归语言模型的指令与对话微调：给模型看“输入是什么、期望怎样回答”的示范，让它提高这些回答的条件概率。

例如，希望模型把用户问题转换成电商售后意图：

```
输入：我的快递三天没更新了。
期望回答：{"intent": "物流查询"}
```

模型学到的应当是：面对类似问题，按要求选择意图并输出指定格式。它还需要在未见过的说法上接受检验，例如“物流轨迹停在转运中心不动了”。

先分清两个维度：

术语	主要描述什么	本文中的例子
预训练	用什么数据、学习什么目标	在大量文本上学习下一个 Token
SFT	用示范数据学习什么行为	给定问题，提高参考回答中各 Token 的概率
全参数微调	更新哪些参数	更新所选模型的全部可训练权重
LoRA	怎样表示和训练权重更新	冻结基座，训练少量低秩矩阵

因此，下面两种组合都成立：

```
SFT + 全参数更新
SFT + LoRA 更新
```

SFT 与 LoRA 不是必须先后执行的两个训练阶段。 一次训练可以直接采用 LoRA 来完成 SFT；已经做过 SFT 的 Instruct 模型，也可以继续在特定任务数据上微调。

常见训练路线可以画成：

```
大量文本 → 预训练 → 基础模型
                      |
            指令 / 问答示范 + SFT
                      |
                      ▼
                 指令 / 对话模型
                      |
              可选：继续做偏好优化

SFT 这一步内部，可以选择全参数微调，也可以选择 LoRA。
```

预训练与这里的 SFT 都可以使用下一个 Token 的交叉熵。主要变化在于数据分布、输入输出格式以及哪些位置参与监督，并不要求换一种 Transformer 结构。

21.2 一条 SFT 数据长什么样？

先看便于人工阅读的指令格式：

```
{
  "instruction": "判断售后意图，只输出包含 intent 字段的 JSON。",
  "input": "我的快递三天没更新了。",
```

```
  "output": "{\"intent\": \"物流查询\"}"
}
```

也可以表达为对话：

```
{
  "messages": [
    {"role": "system", "content": "你是电商售后分流助手。"},
    {"role": "user", "content": "判断售后意图，只输出包含 intent 字段的 JSON。我的快递三天没更新了。"},
    {"role": "assistant", "content": "{\"intent\": \"物流查询\"}"}
  ]
}
```

这两段是同一教学任务的不同表示。实际框架需要相应的数据字段映射；把 JSON 文件直接交给模型，并不意味着模型会自动理解哪个字段是监督目标。

Tokenizer 最终处理的是按模板组织的序列。示意如下，尖括号内容只是教学占位符：

```
<system> 你是电商售后分流助手。
<user> 判断售后意图……我的快递三天没更新了。
<assistant> {"intent": "物流查询"} <结束标记>
```

角色标记、换行和结束标记也是格式的一部分。 应使用与模型匹配的 Chat Template，训练和推理保持一致。对于完整的训练对话，通常不再额外追加一个空的 assistant 开头；推理时才需要在用户问题后准备让模型开始回答。具体序列化与 `add_generation_prompt` 的含义见 Transformers Chat Templates 文档。

多轮对话中，可以监督所有 assistant 回答，也可以只监督最后一轮。必须根据训练配置检查实际生成的 labels，不能仅凭文件里有 `messages` 就认定只训练回答。

21.3 SFT 的损失：给定问题，让参考回答更可能出现

设问题及其模板前缀为 x，参考回答为：

$$y = (y_1, y_2, \ldots, y_m)$$

自回归分解为：

$$p_\theta(y \mid x) = \prod_{t=1}^{m} p_\theta(y_t \mid x, y_{<t})$$

对单条样本，采用回答 Token 平均的负对数似然：

$$\boxed{\mathcal{L}_{\mathrm{SFT}} = -\frac{1}{m} \sum_{t=1}^{m} \log p_\theta(y_t \mid x, y_{<t})}$$

这里，m 包含本次选择监督的回答结束标记。对多个样本，本章采用“所有有效目标 Token 的损失之和，除以有效目标 Token 总数”；它与“先对每条回答平均、再对样本平均”在回答长度不同时并不相同。

直觉是：模型给正确 Token 的概率越高，这一项的损失越小。

正确 Token 的概率 p	**负对数损失 $-\ln p$**
0.1	约 2.3026
0.5	约 0.6931
0.9	约 0.1054

假设回答简化为两个 Token，再加一个结束 Token：

```
好 → 呀 → <EOS>
```

模型在相应位置给正确 Token 的概率分别为：

$$0.5, \quad 0.25, \quad 0.8$$

那么：

$$\mathcal{L} = -\frac{\ln 0.5 + \ln 0.25 + \ln 0.8}{3} = -\frac{\ln 0.1}{3} \approx 0.7675$$

这三个概率并不是模型把整句自由生成完之后再打分得到的。训练时每个位置都给定正确的前文，例如预测“呀”时，条件中已经有参考答案里的“好”。这种方式称为 **Teacher Forcing**。

21.4 回答已经放进输入，为什么不是偷看答案？

因为自回归模型采用 Causal Mask：一个位置只能看到自己及更早的位置，而它对应的预测目标是**下一个** Token。

把上一节的样本继续简化，假设每个格子恰好是一个 Token：

```
位置：          0       1          2           3     4      5       6
input_ids：  <BOS>     问    <assistant>     好     呀    <EOS>   <PAD>
Token ID：      0       1          2           3     4      5       6
```

本例只监督回答和结束标记：

```
原位置 labels：
            -100    -100    -100          3     4      5     -100
```

模型输出的 `logits[:, t, :]` 用来预测 `input_ids[:, t + 1]`。因此计算损失时配对为：

用哪个位置的输出	这个位置能看到的最后一个 Token	预测目标	是否计入损失
位置 0	<BOS>	问	否
位置 1	问	<assistant>	否
位置 2	<assistant>	好	是
位置 3	好	呀	是
位置 4	呀	<EOS>	是
位置 5	<EOS>	<PAD>	否

预测“好”的是位置 2 的输出；这个位置看不到位置 3 的“好”。这就是答案放进序列却不泄漏给自身预测的原因。

训练时可以在一次前向传播中并行计算这些位置。推理时没有参考回答，才需要把模型刚生成的 Token 接回输入，逐步生成后续内容。

常见代码对应：

```
# logits: [batch_size, sequence_length, vocabulary_size]
# labels: [batch_size, sequence_length]，此时仍与 input_ids 原位置对齐
shift_logits = logits[:, :-1, :]
shift_labels = labels[:, 1:]
```

错位只做一次。 使用会在内部计算因果语言模型损失的接口时，通常直接传入原位置 labels；如果自己计算交叉熵，才按上面的方式配对。不要先手工移位，再交给会再次移位的模型接口。

21.5 三种 Mask 各管什么？

名称	控制什么	本例中的作用
Causal Mask	当前位置能否看未来位置	预测“好”的位置不能看到“好”及后面的内容
Padding Attention Mask	哪些补齐位置不应作为有效上下文	将补齐的 <PAD> 标为无效
Loss Mask / labels=-100	哪些目标不计入损失	忽略问题、角色前缀和 Padding 的预测目标

本例的 Padding Attention Mask 为：

```
attention_mask = [1, 1, 1, 1, 1, 1, 0]
```

而 labels 为：

```
labels = [-100, -100, -100, 3, 4, 5, -100]
```

这里 `-100` 是交叉熵接口采用的忽略值，不是词表里的 Token ID。**不要把 input_ids 中的问题也替换成 -100**；问题仍然是模型回答所需的条件。

“不计算问题部分的预测损失”也不等于“问题不参与反向传播”。回答的输出依赖问题形成的隐藏状态，因此回答损失仍可以沿这些计算路径传播。

还要留意两个细节：

- 如果 Padding 恰好复用了 EOS 的 ID，不能把所有等于该 ID 的标签都屏蔽；应根据实际补齐位置屏蔽，保留回答中真正需要学习的结束标记。
- 如果截断后回答全被删掉，有效目标数就是 0。这种样本应重新处理或过滤；全部目标被忽略时，不能期待平均损失仍是一个正常的训练信号。

SFT 框架对全文、completion 和 assistant 部分的监督策略可能不同。例如 TRL 的 assistant-only 监督需要模板能够提供相应边界信息，应显式核对配置与生成的掩码。可对照 TRL SFTTrainer 的监督范围说明。

21.6 从数据到一次参数更新

把前面的步骤连起来：

```
清洗并划分示范数据
        ↓
套用 Chat Template → Tokenize
        ↓
构造 input_ids、attention_mask、labels
        ↓
前向传播 → 各位置的词表 logits
        ↓
错位配对 → 只对有效目标计算交叉熵
        ↓
反向传播 → 得到可训练参数的梯度
        ↓
梯度累积 / 裁剪（按训练配置）
        ↓
Optimizer 更新 → 清空本轮梯度
        ↓
定期验证、保存 → 下一批数据
```

用最简单的梯度下降表示：

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta \mathcal{L}$$

η 是学习率。实际训练常使用 AdamW 等优化器，更新还取决于梯度的历史统计量，因此不能把上述公式当成 AdamW 的完整实现。

在全参数 SFT 中，θ 包含模型全部参与训练的参数；在 LoRA SFT 中，主要改为各目标层的 A, B。**参考答案、因果预测方式和 SFT 损失都可以保持相同。**

注意 `model.train()` 的含义是切换训练模式，例如启用 Dropout；它本身既不计算梯度，也不更新权重。真正的更新需要损失、`backward()` 和优化器。

21.7 用 PyTorch 验证一次损失与梯度

下面的完整示例只需要 CPU 和 PyTorch。它直接构造词表 logits，验证第 21.3 ~ 21.5 节的算术和掩码；没有加载语言模型，也没有把这几个数当作真正的 LLM 微调实验。

为看清梯度，示例把 logits 临时当作可优化变量。真实训练中它们是模型的输出，梯度还要继续传回模型参数。

```
import math
import torch
import torch.nn.functional as F

# 词表：<BOS>、问、<assistant>、好、呀、<EOS>、<PAD>
input_ids = torch.tensor([[0, 1, 2, 3, 4, 5, 6]])
labels = input_ids.clone()
labels[:, :3] = -100        # 不监督问题与回答前缀
labels[:, 6:] = -100        # 不监督 Padding
vocab_size = 7

logits = torch.zeros(1, 7, vocab_size, dtype=torch.float64)
for position, target_id, probability in [
    (2, 3, 0.5),            # <assistant> 后预测“好”
    (3, 4, 0.25),           # “好”后预测“呀”
    (4, 5, 0.8),            # “呀”后预测 EOS
]:
    other = (1.0 - probability) / (vocab_size - 1)
    logits[0, position, :] = math.log(other)
    logits[0, position, target_id] = math.log(probability)
logits.requires_grad_()

def answer_loss(scores):
    targets = labels[:, 1:].reshape(-1)
    assert (targets != -100).any(), "至少需要一个有效目标 Token"
    return F.cross_entropy(
        scores[:, :-1, :].reshape(-1, vocab_size),
        targets,
        ignore_index=-100,
    )

optimizer = torch.optim.SGD([logits], lr=0.1)
optimizer.zero_grad(set_to_none=True)
loss = answer_loss(logits)
loss.backward()

assert torch.allclose(
```

```
    loss.detach(), torch.tensor(-math.log(0.1) / 3, dtype=torch.float64)
)
assert logits.grad[0, [0, 1, 5, 6], :].abs().sum().item() == 0
assert logits.grad[0, 2:5, :].abs().sum().item() > 0

before = loss.item()
optimizer.step()
with torch.no_grad():
    after = answer_loss(logits).item()
assert after < before
print(f"更新前 loss = {before:.4f}")   # 约 0.7675
print(f"更新后 loss = {after:.4f}")
```

`F.cross_entropy` 接收未归一化的 logits，内部完成相应的对数概率计算；不要先对 logits 做 Softmax 再当成 logits 传入。`ignore_index` 的平均只覆盖未忽略的目标，接口定义见 PyTorch CrossEntropyLoss 文档。

这里断言为 0 的是"被忽略预测位置的 logits 梯度"，不是实际 Transformer 中这些位置的全部隐藏状态梯度。实际模型仍会通过上下文依赖传递梯度。

21.8 Batch、梯度累积与训练轮数

这几个参数描述不同的量：

参数	含义	一个教学例子
Micro-batch Size	每张卡一次前向 / 反向处理的样本数	1
Gradient Accumulation Steps	累积多少个 Micro-batch 后更新一次	8
数据并行卡数	各自处理不同样本并汇总梯度的卡数	1
Epoch	训练集被遍历的轮数	3
Learning Rate	控制每次更新的步长尺度	由验证结果选择

在普通数据并行、每批样本数一致的情况下：

$$B_{\text{effective}} = B_{\text{micro}} \times N_{\text{accumulation}} \times N_{\text{data parallel}}$$

因此：

$$1 \times 8 \times 1 = 8$$

这表示一次参数更新汇集约 8 条样本，而不是显存里必须同时放下 8 条样本。张量并行或流水线并行使用的卡数不能直接代入这里的数据并行倍数。

梯度累积时，每个 Micro-batch 通常都执行反向传播，到了累积边界才执行优化器更新。**不要在每个 Micro-batch 前都清空已经累积的梯度。**

若手写训练循环，还要处理损失归一化：各批有效目标数相等时，常见做法是将批损失除以累积步数；长度差异明显时，要按有效目标 Token 总数加权，才能等价于本章定义的大批次 Token 平均损失。最后不足一个累积窗口的部分也需要正确处理。

Epoch 更多不保证效果更好。训练损失持续下降、验证损失反而上升时，通常需要检查过拟合，而不是继续增加轮数。

21.9 怎样判断 SFT 训练有效?

在训练前划分训练、验证、测试数据。同一条对话、同一来源的高度相似改写，应尽量放在同一侧，避免模型在测试时遇到训练内容的近似副本。

以售后分流为例，需要同时观察：

观察项	能回答什么	不能单独证明什么
训练 Loss	是否拟合训练示范	是否能处理新问题
验证 Loss	在同一模板、同一监督规则下是否改善	自由生成是否总是符合格式
JSON 有效率	回答能否被正确解析	意图是否判断正确
意图准确率	分类是否命中参考意图	回复内容是否充分、可靠
微调前后固定题目对比	行为具体发生了什么变化	对所有真实用户都有效

比较时使用相同的测试题、模板和生成设置，并设置足够的生成长度。训练中使用参考前文，推理中使用模型自己的输出，因此 Token 准确率与整段回答的质量并不等价。

常见异常可以按下面顺序检查：

- **Loss 一开始就异常小或为 NaN：** 查看有效 label 数量、是否重复错位、回答是否被截断、数值是否溢出。
- **模型复述问题而没有回答：** 查看监督范围、角色模板和推理前缀。
- **回答一直不结束：** 查看结束标记是否进入监督、推理端是否使用匹配的停止条件。
- **训练集效果很好，测试集变差：** 检查数据量、标签一致性、过拟合，以及训练任务是否过窄。
- **显存不足：** 先检查序列长度和 Micro-batch，再考虑梯度检查点、精度与 LoRA / QLoRA；LoRA 的显存边界见第 22 章。

21.10 三道练习与答案

练习 1： 一条样本中有 10 个前缀 Token、4 个回答 Token、1 个需监督的结束 Token和 3 个 Padding Token。仅监督回答时，损失平均的分母是多少?

答案： 5。前缀和 Padding 不计入目标数；结束标记是本题明确要求学习的目标。

练习 2： 把训练从全参数 SFT 改为 LoRA SFT，是否必须把交叉熵换成另外一种损失?

答案： 不必。改变的是参与更新的参数及其表示方式，训练目标可以完全相同。

练习 3： 每张卡一次处理 2 条样本，梯度累积 4 次，采用 2 张数据并行卡。一次更新的有效 Batch Size 是多少?

答案： $2 \times 4 \times 2 = 16$ 条样本。它是样本数量，不是有效目标 Token 数；回答长度不同时，两者要分别统计。

> 本章要记住：SFT 用示范回答建立监督，因果错位决定“预测谁”，Loss Mask 决定“哪些目标算分”，优化器决定“怎样更新可训练参数”。下一章只更换参数更新方式，把这条链路接到 LoRA。

22. LoRA 训练：冻结基座以后，小矩阵到底怎样学？

上一章讲的是 SFT 的数据和训练目标；本章沿用这个目标，改为只更新 LoRA 参数。**SFT 回答“用什么示范学习”，LoRA 回答“让哪些参数学习”。**

第 13 章已经介绍低秩分解。本章把公式接到反向传播、优化器和模型保存，最后用一个不需要下载大模型的 CPU 实验验证。

22.1 一次 LoRA SFT 训练经过哪些步骤？

```
问题 + 标准回答
    ↓ 使用基座对应的 Chat Template
input_ids、attention_mask、labels
    ↓ 基座权重 W 冻结，插入的 A、B 参与计算
所有位置的词表 logits
    ↓ 下一 Token 对齐，只统计指定监督位置
交叉熵 loss
    ↓ backward()
计算 A、B 的梯度
    ↓ optimizer.step()
更新 A、B，W 保持不变
```

因此，把全量 SFT 换成 LoRA SFT，不需要把交叉熵换成一种“LoRA 损失”。变化主要发生在参数化方式和可训练参数集合。

以下讨论普通 LoRA：基座和 bias 都冻结，仅训练各目标线性层的 A, B。若另外开放词嵌入、输出头或 bias，就必须把它们也计入可训练参数和保存内容。

22.2 前向传播：原来的答案，加上正在学习的修正

沿用第 13 章的列向量写法，暂时令 dropout 为 0：

$$y = Wx + sBAx, \qquad s = \frac{\alpha}{r}$$

其中：

$$x \in \mathbb{R}^{d_{in}}, \quad W \in \mathbb{R}^{d_{out} \times d_{in}}, \quad A \in \mathbb{R}^{r \times d_{in}}, \quad B \in \mathbb{R}^{d_{out} \times r}$$

可以把 LoRA 分支拆成两步：

$$z = Ax \in \mathbb{R}^{r}, \qquad \Delta y = sBz \in \mathbb{R}^{d_{out}}$$

不是先把整个大矩阵 BA 算出来才能训练；实际前向可以依次计算两个小线性变换。

原始 LoRA 采用冻结基座、训练低秩增量的形式，并使用 α/r 缩放。本章后续数字是为教学自拟的例子。
LoRA 原论文，第 4.1 节

22.3 反向传播：冻结权重，不等于切断计算图

令上游传回来的梯度为：

$$g = \frac{\partial L}{\partial y} \in \mathbb{R}^{d_{out}}$$

由链式法则可以得到：

$$\frac{\partial L}{\partial B} = s\, g(Ax)^\top$$

$$\frac{\partial L}{\partial A} = s\, B^\top g\, x^\top$$

如果输入 x 来自更前面的可训练部分，还需要继续传播：

$$\frac{\partial L}{\partial x} = W^\top g + sA^\top B^\top g$$

注意其中依然出现了 W。**不更新 W，不代表反向传播可以跳过 Wx 对输入的影响。**

在 PyTorch 中，冻结基座参数通常使用 `parameter.requires_grad_(False)`。不要为了冻结权重，就把整个模型前向放进 `torch.no_grad()`，那会连 LoRA 所需的梯度一起切断。

即使只把某个冻结 Linear 包进 `no_grad()`，当它的输入连接着更早的可训练 Adapter 时，也会丢掉经该分支传向前层的梯度。应让自动求导根据张量依赖决定需要保留哪些计算。

还要区分三件事：`requires_grad=False` 控制参数梯度，`model.eval()` 改变 dropout 等层的行为，`no_grad()` 禁止记录所包裹运算的梯度图；三者不能互相替代。

22.4 为什么通常 A 随机初始化、B 初始化为 0?

常见初始化是：

$$A \neq 0, \qquad B = 0$$

于是训练开始时：

$$\Delta W = sBA = 0, \qquad y = Wx$$

新加的分支起初不会改变基座输出。PEFT 的默认普通 LoRA 初始化采用 A 的 Kaiming 均匀初始化和 B 的零初始化；其他初始化方法可以不同。PEFT LoRA：Initialization

把 $B = 0$ 代入上一节梯度公式：

$$\frac{\partial L}{\partial A} = 0, \qquad \frac{\partial L}{\partial B} = s\, g(Ax)^\top$$

只要损失梯度和输入投影合适，B 在第一步就能学；B 更新后，后续步骤通常也能给 A 提供非零梯度。

如果把 A、B **同时设为 0**，两个矩阵的数据损失梯度都会为 0，普通梯度下降无法启动这条分支。

这里说的是损失产生的梯度。若优化器包含权重衰减等额外机制，“梯度为 0”不必然等于“参数数值绝对不变”。下面用不带权重衰减和动量的 SGD 手算。

22.5 完整手算：一个样本，一次 SGD 更新

为方便手算，把真实 LLM 的交叉熵换成一个小回归问题。取：

$$W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad t = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$r = 1, \quad \alpha = 1, \quad s = 1, \quad A = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

这里人为指定 A 的非零初值，让数字容易计算。

第一步：前向与损失。

$$Ax = 1, \qquad y = Wx + BAx = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

定义半平方误差：

$$L = \frac{1}{2}\|y - t\|_2^2 = \frac{1}{2}\left[(-1)^2 + 2^2\right] = 2.5$$

第二步：计算梯度。

$$g = y - t = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

$$\frac{\partial L}{\partial B} = g(Ax)^\top = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, \qquad \frac{\partial L}{\partial A} = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

第三步：学习率取 $\eta = 0.1$，同时更新参数。

$$B_{new} = B - \eta \frac{\partial L}{\partial B} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}, \qquad A_{new} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

第四步：重新前向，检查有没有改进。

$$y_{new} = \begin{bmatrix} 1.1 \\ 1.8 \end{bmatrix}, \qquad L_{new} = \frac{1}{2}(0.9^2 + 1.8^2) = 2.025$$

W 完全没动，损失已经从 2.5 降到了 2.025。

下一次反向时，A 的梯度变成：

$$B_{new}^\top (y_{new} - t) x^\top = (-0.09 - 0.36)\begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} -0.45 & -0.9 \end{bmatrix}$$

这说明“第一步 A 梯度为零”与“以后 A 不能学习”是两回事。

22.6 CPU 小实验：让 A、B 学会一个低秩修正

下面是一段完整的 PyTorch 程序，只用 CPU，不加载 Tokenizer，不下载模型。它先验证上一节手算，再用多组输入训练一个 LoRA Linear，最后检查合并前后输出是否一致。

这段程序演示层级参数更新，并不是完整的 LLM SFT。 手算用半平方误差，多样本训练用均方误差 MSE；真实 SFT 则使用上一章的下一 Token 交叉熵。

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(7)


class LoRALinear(nn.Module):
    def __init__(self, weight, rank=1, alpha=1.0, dropout=0.0):
        super().__init__()
        out_features, in_features = weight.shape
        if rank < 1:
            raise ValueError("rank must be positive")
        self.base = nn.Linear(in_features, out_features, bias=False)
        with torch.no_grad():
            self.base.weight.copy_(weight)
        self.base.weight.requires_grad_(False)
        self.A = nn.Parameter(torch.empty(rank, in_features))
        self.B = nn.Parameter(torch.zeros(out_features, rank))
        nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))
        self.scale = alpha / rank
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # PyTorch 按行存放样本：F.linear(x, A) = x @ A.T
        correction = F.linear(F.linear(self.dropout(x), self.A), self.B)
        return self.base(x) + self.scale * correction

    def merged_weight(self):
        return self.base.weight + self.scale * (self.B @ self.A)


# 一、精确复现手算：每行是一个样本。
layer = LoRALinear(torch.eye(2))
with torch.no_grad():
    layer.A.copy_(torch.tensor([[1.0, 0.0]]))
x = torch.tensor([[1.0, 2.0]])
target = torch.tensor([[2.0, 0.0]])
optimizer = torch.optim.SGD(
    [p for p in layer.parameters() if p.requires_grad], lr=0.1
)
optimizer.zero_grad(set_to_none=True)
loss = 0.5 * (layer(x) - target).square().sum()
loss.backward()
assert layer.base.weight.grad is None
torch.testing.assert_close(layer.A.grad, torch.zeros_like(layer.A))
torch.testing.assert_close(layer.B.grad, torch.tensor([[-1.0], [2.0]]))
optimizer.step()
```

```
with torch.no_grad():
    after = 0.5 * (layer(x) - target).square().sum()
torch.testing.assert_close(after, torch.tensor(2.025))
print(f"手算验证：{loss.item():.3f} -> {after.item():.3f}")
optimizer.zero_grad(set_to_none=True)
next_loss = 0.5 * (layer(x) - target).square().sum()
next_loss.backward()
torch.testing.assert_close(layer.A.grad, torch.tensor([[-0.45, -0.9]]))

# 二、重新创建模型，使用随机 A 和全零 B 学一个已知的 rank-1 修正。
layer = LoRALinear(torch.eye(2))
base_before = layer.base.weight.detach().clone()
inputs = torch.tensor([
    [1.0, 0.0], [0.0, 1.0], [1.0, 1.0],
    [1.0, -1.0], [-1.0, 0.0], [0.0, -1.0],
])
true_A = torch.tensor([[1.0, 2.0]])
true_B = torch.tensor([[0.5], [-0.25]])
targets = F.linear(inputs, base_before + true_B @ true_A)
trainable = [p for p in layer.parameters() if p.requires_grad]
print("可训练参数：", [(n, tuple(p.shape))
                        for n, p in layer.named_parameters() if p.requires_grad])
print("可训练参数个数：", sum(p.numel() for p in trainable))
optimizer = torch.optim.Adam(trainable, lr=0.03)
layer.train()
initial_loss = F.mse_loss(layer(inputs), targets).item()
for step in range(400):
    optimizer.zero_grad(set_to_none=True)
    loss = F.mse_loss(layer(inputs), targets)
    loss.backward()
    optimizer.step()

# 三、验证基座冻结、损失下降，以及普通 LoRA 的合并公式。
layer.eval()
with torch.no_grad():
    final_loss = F.mse_loss(layer(inputs), targets).item()
    separate_output = layer(inputs)
    merged_output = F.linear(inputs, layer.merged_weight())
assert torch.equal(layer.base.weight, base_before)
assert layer.base.weight.grad is None
assert final_loss < initial_loss * 0.1
torch.testing.assert_close(separate_output, merged_output, rtol=1e-5, atol=1e-6)
print(f"回归 MSE：{initial_loss:.6f} -> {final_loss:.6f}")
print("基座未更新；合并前后输出一致。")
```

这个 2×2 玩具层有 4 个基座参数、4 个 LoRA 参数，只为让每个数字可见，并不展示参数节省。换成第 13 章的 4096×4096、rank=8，才体现“大矩阵配小增量”的规模优势。

代码把实际优化器限制为 `requires_grad=True` 的参数。真实模型训练前也应打印可训练参数的名称、数量和比例，确认目标层确实插入了 Adapter，且没有意外开放全部基座。

22.7 rank、alpha、dropout、target 和学习率怎么区分？

参数	直接改变什么	课堂上应该观察什么
`rank` / `r`	A、B 的形状及增量的最大秩	参数量、训练开销、验证集表现
`lora_alpha`	普通 LoRA 的分支缩放 $s = \alpha/r$	相同 A、B 对输出的影响幅度
`lora_dropout`	训练时对 LoRA 分支输入施加 dropout	泛化与训练稳定性；推理时关闭
`target_modules`	哪些线性层具有可训练增量	实际匹配到的模块和参数总量
`learning_rate`	优化器每步怎样更新参数	收敛速度、震荡、验证损失

例如 $r = 8, \alpha = 16$ 时 $s = 2$；如果只把 rank 改成 16，保持 alpha=16，则 $s = 1$，同时参数量也翻倍。

若要研究 rank 的影响，可以保持 α/r 不变，减少缩放变化带来的干扰。但初始化、表达能力和优化过程仍发生变化，不能保证不同 rank 的训练轨迹相同。

Alpha 出现在前向公式中，学习率出现在参数更新公式中；两者会共同影响训练，但一般不能机械地互换。上面的 `lr=0.03` 是这个小回归实验的设置，不能直接搬到大模型 SFT。

`q_proj`、`v_proj` 等名字取决于具体模型实现；GQA 中 K/V 的输出维度也可能小于 Q。统计参数应逐个目标矩阵求和：

$$N_{LoRA} = \sum_{\ell \in \text{目标层}} r_\ell \left(d_{in,\ell} + d_{out,\ell}\right)$$

本章使用普通 LoRA 的 α/r。PEFT 还支持 rsLoRA 等变体，例如 rsLoRA 使用 $\alpha/\sqrt{r}$，阅读配置时要先辨别具体方法。PEFT LoRA 配置说明

22.8 显存为什么下降？为什么仍然可能 OOM?

训练显存可以粗略拆成：

```
基座权重 + Adapter 权重
+ 需要保存的激活
+ 可训练参数的梯度和优化器状态
+ 临时张量与框架开销
```

LoRA 主要省下大部分基座参数对应的梯度和优化器状态。**基座权重仍需参与前向，反向传播所需的激活也没有自动消失。**

例如 7B 参数仅按每参数 2 字节计，基座权重约为 14 GB（十进制），还没有计入上述其他项目；因此不能从 Adapter 很小推导出任意显卡都能训练。

出现 OOM 时，要结合瓶颈调整单卡 micro-batch、序列长度、梯度检查点和精度。梯度累积通过多次小批次实现更大的有效 batch，并不消除单次前向所需的基座和激活空间。

QLoRA 进一步让冻结基座以低比特形式存储，并通过它反传来训练 LoRA Adapter；原论文包含 4-bit NF4、双重量化和分页优化器设计。它不是“把 A、B 都变成 4-bit 再训练”。QLoRA 原论文

低比特存储也不等于所有计算都用同一种低比特精度。要分别看基座的存储形式、计算 dtype、Adapter dtype，以及当前硬件和实现是否支持。

22.9 训练以后：保存、加载和合并是什么关系？

保存 Adapter：保存 A、B 以及 rank、alpha、目标模块等配置。PEFT 的 `save_pretrained()` 默认保存适配器相关内容，加载时仍需要基座；`modules_to_save` 等配置还可能让额外模块进入产物。PEFT checkpoint 格式

加载 Adapter：先加载匹配的基座，再挂载适配器。不要只核对模型名字；应记录准确的基座版本或 revision，以及 Tokenizer、Chat Template 和词表变动。相同形状的另一个基座，可能能成功加载权重，但得到错误的任务行为。

合并权重：对本章的普通 LoRA，在关闭 dropout 的推理状态下，可以形成：

$$W_{merged} = W + \frac{\alpha}{r} BA$$

这样推理时直接使用 $W_{merged}x$，无需保留独立的两层 LoRA 分支。合并后的矩阵与原 W 形状相同，LoRA 没有把基座变成一个更小的模型。

PEFT 提供 `merge_and_unload()` 返回合并后的普通模型。合并后若想方便地切换 Adapter，应另行保留原始基座和 Adapter 文件；量化模型与部分方法的合并还取决于实现支持。PEFT：合并权重与限制

浮点舍入和量化可能带来数值差异，应在相同输入、相同模板和相同生成设置下比较合并前后的输出。用于继续训练的 checkpoint 通常还需要优化器、调度器等状态；单独的 Adapter 不能恢复原有训练进度。

22.10 衔接已有的 LLaMA-Factory 实训案例

本目录已有 LLaMA-Factory 案例包，可以把上述原理接到真实模型训练。使用其中的 LoRA SFT 配置，模型为 `Qwen/Qwen2.5-1.5B-Instruct`，任务是电商售后意图分类与回复生成。

这是一个已经具备指令能力的模型继续做任务 SFT 的例子。案例只有 40 条训练候选数据，适合观察流程；不能仅凭这次小实验判断真实业务效果。

先读懂配置中的两个关键选择

下面是关键字段的节选，不是独立的完整配置：

```yaml
stage: sft
finetuning_type: lora
lora_rank: 8
lora_alpha: 16
lora_dropout: 0.05
lora_target: all

dataset: ecommerce_intent
template: qwen
cutoff_len: 512

per_device_train_batch_size: 1
gradient_accumulation_steps: 8
learning_rate: 5.0e-5
```

```
num_train_epochs: 3.0
val_size: 0.2
```

`stage: sft` 选择监督微调，`finetuning_type: lora` 选择低秩参数更新。`lora_target: all` 表示按框架规则匹配可应用 LoRA 的线性层，不代表把所有基座参数都开放训练；具体排除项和模块匹配以所用版本为准。LLaMA-Factory SFT 说明、目标模块选择实现

为让课堂中的监督范围更清楚，可以在复制出的实验配置中显式补上：

```
train_on_prompt: false
mask_history: false
```

前者屏蔽 prompt 的预测损失，保留它作为输入；后者允许多轮中的历史 assistant 回答也参与监督。这个案例是单轮数据，不涉及历史轮次的选择。LLaMA-Factory 监督标签构造

再按明确的工作目录运行

1. 在**案例包根目录**检查数据：

   ```
   python scripts/validate_dataset.py
   ```

2. 将案例包 `data/ecommerce_intent_sft.json` 复制到已安装的 **LLaMA-Factory 仓库** `data/` **目录**。将 `data/dataset_info_fragment.json` 中的 `ecommerce_intent` 条目合并到该仓库的 `data/dataset_info.json`，保留原有条目。独立测试集不并入训练数据。

3. 将完整的 `configs/qwen25_ecommerce_lora_sft.yaml` 复制到该仓库的 `examples/course/qwen25_ecommerce_lora_sft.yaml`，目录不存在时先创建。检查模型路径、精度及输出目录；案例默认启用 FP16，硬件支持 BF16 时可改为 BF16，二者不要同时启用。

4. 在 **LLaMA-Factory 仓库根目录**启动训练。下面用命令覆盖本次评估、保存间隔和输出目录，便于观察短实验：

   ```
   llamafactory-cli train examples/course/qwen25_ecommerce_lora_sft.yaml
   eval_steps=2 save_steps=4 output_dir=saves/course_sft_lora_run1
   ```

每次新实验换一个输出目录。实际执行前需准备好支持该配置的训练环境和模型文件；上述命令会进行真实模型训练，和第 21.7、22.6 节的 CPU 数值演示不同。

为什么需要解释评估和保存的步数？

在单卡、无样本过滤、无 Packing 的假设下：

$$40 \times (1 - 0.2) = 32 \text{ 条训练样本}, \qquad 8 \text{ 条验证样本}$$

$$\text{每轮更新次数} = \frac{32}{1 \times 8} = 4, \qquad \text{总更新次数} = 4 \times 3 = 12$$

原配置 `eval_steps: 10` 会在第 10 次优化器更新时周期评估；`save_steps: 20` 在这 12 次更新中到不了周期触发点。上面的命令改成每 2 次更新评估、每 4 次更新保存，便于课堂观察。训练结束仍会保存最终适配器，末步是否另存 checkpoint 或额外评估以安装版本的日志为准。LLaMA-Factory SFT 保存流程

`max_samples: 1000` 只是样本数上限，不会把现有 40 条样本扩充成 1000 条。40 条数据的验证划分也不能保证每类都有足够样本，正式实验应准备更充分、划分合理的数据。

训练后比较什么?

先用基座回答案例包中的 10 条独立测试问题，再加载**同一基座与本次适配器**回答相同问题。保持模板和生成参数一致，按案例包 `scripts/score_predictions.py` 顶部说明，将每次预测分别放入案例包根目录的 `predictions.json`，运行：

```
python scripts/score_predictions.py
```

记录两次的 JSON 有效率与意图准确率，并保留各自的预测副本。评分脚本需要事先生成的预测文件，本身不会调用模型。案例要求输出 `intent` 和 `reply` 两个字段；还应人工抽查回复是否与问题匹配。

训练通常不需要生成阶段那种跨 Decode 步复用的 KV Cache。LLaMA-Factory 在支持的梯度检查点训练路径中会关闭模型的 `use_cache`；梯度检查点通过反向时重算部分激活来节省显存，不是把激活当成推理 Cache 保存。梯度检查点实现

22.11 三道练习：从参数更新回到 SFT

练习 1： 一个无 bias 的线性层为 1024×4096，rank=8，alpha=16。LoRA 参数有多少？缩放系数是多少?

答案： $8(4096 + 1024) = 40,960$ 个，缩放为 $16/8 = 2$。冻结基座有 4,194,304 个参数；Adapter 参数约为该基座矩阵参数的 0.98%。

练习 2： 第一轮反向发现 A 梯度全零、B 梯度非零，是否一定训练失败？若两者持续全零，应先看什么?

答案： 在 A 非零、B 零初始化下，第一轮这种现象正常。若持续全零，应检查是否同时零初始化、是否使用了 `no_grad()` 或 `detach()` 切断计算图、目标模块是否被用到、数据是否具有有效监督位置，以及优化器是否包含 Adapter 参数。

练习 3： 上一章 labels 的 prompt 位置标为 `-100`，采用 LoRA 后应不应该取消 mask，让 A、B 能看到问题?

答案： 不需要。Label mask 决定哪些位置计入损失，不负责从输入中删除问题。模型仍通过输入和因果注意力读取问题；回答位置的交叉熵可以把梯度传给影响回答预测的 LoRA 参数。

> **LoRA SFT 保留 SFT 的监督目标，让梯度主要更新低秩 Adapter；省下的主要是可训练参数相关开销，基座计算、数据质量和评测依然重要。**